5 - RECURRENT NEURAL NETWORKS

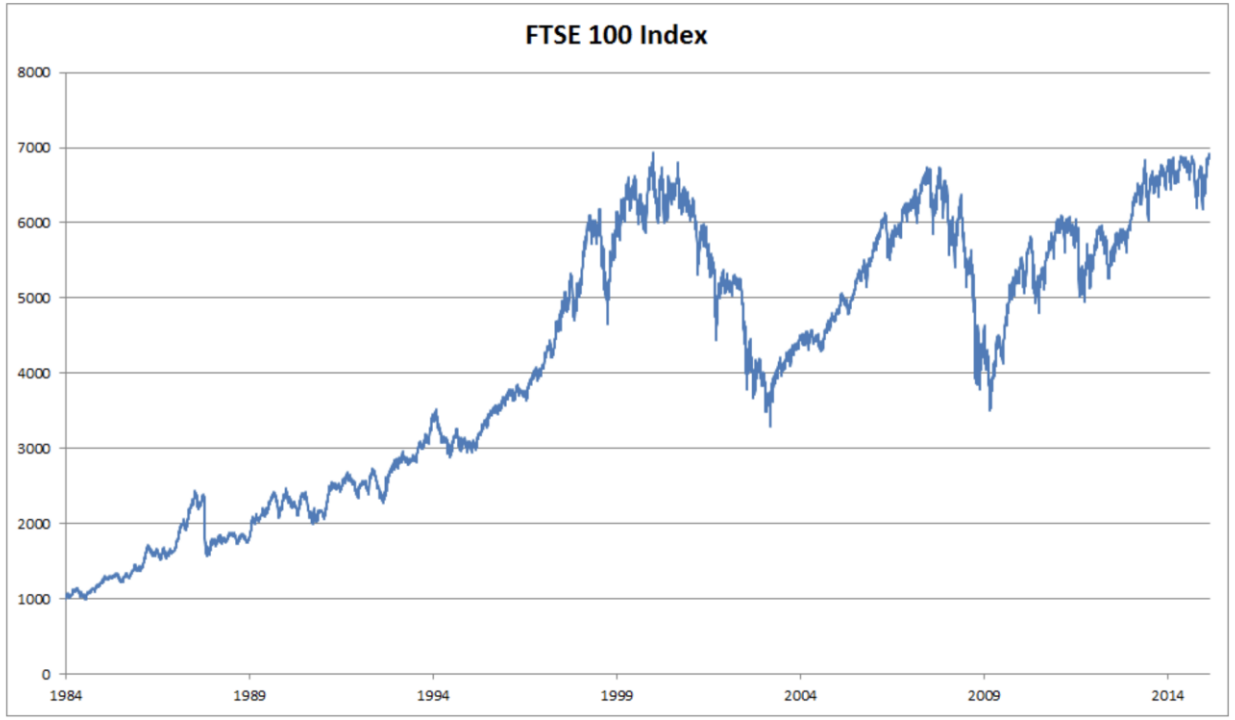
OSCAR LLORENTE GONZALEZ

OLLORENTE@COMILLAS.EDU

INDEX

1 - BEFORE RNNS

AUTOREGRESSIVE MODELS

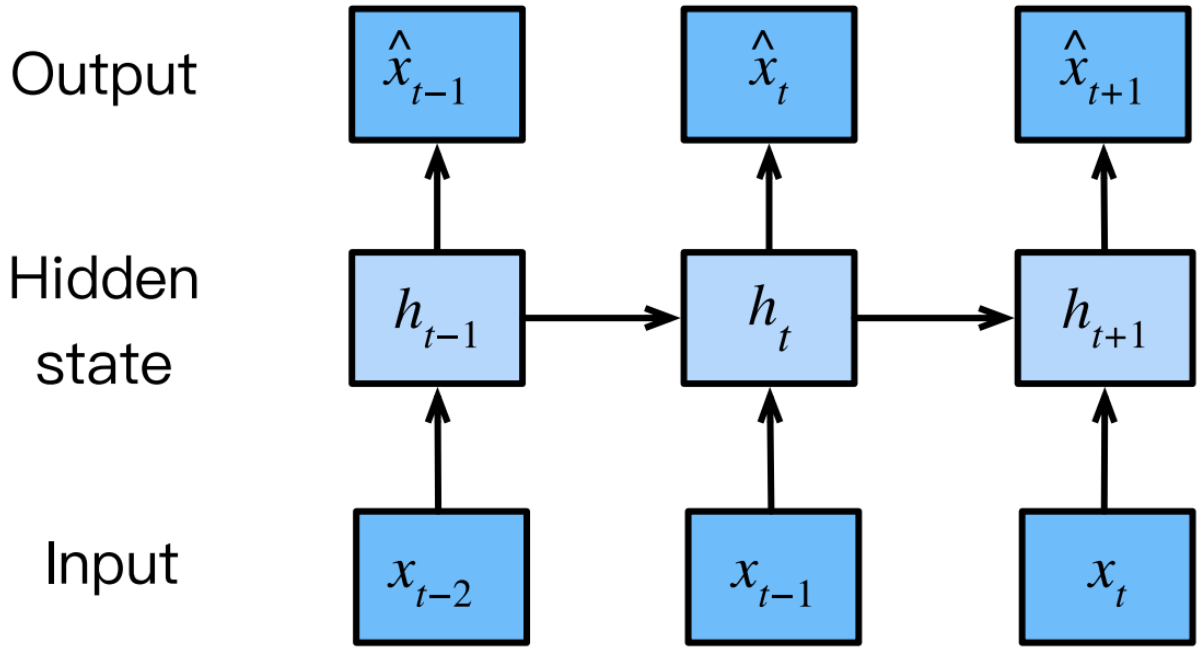


FTSE 100 index over about 30 years.

$$P(x_t \mid x_{t-1}, \ldots, x_1)$$

$$\mathbb{E}[(x_t \mid x_{t-1}, \ldots, x_1)],$$

LATENT AUTOREGRESSIVE MODEL



SEQUENCE MODELS

$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1}, \ldots, x_1).$$

MARKOV MODELS

- Markov condition: that the future is conditionally independent of the past, given the recent history

$$\longrightarrow \quad x_{t-1}, \ldots, x_{t-\tau}$$

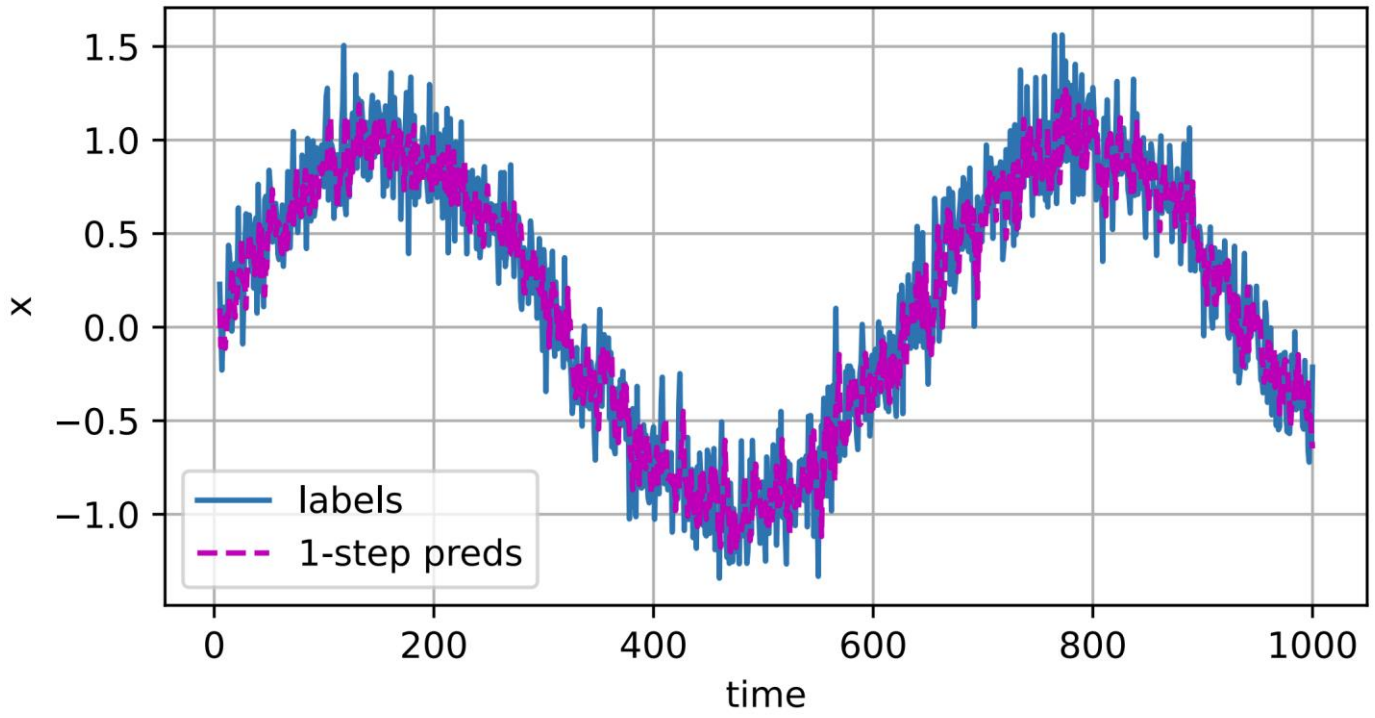
$$P(x_1, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_{t-1}).$$

ORDER OF DECODING

- Factorizing text in the same direction in which we read it is preferred for the task of language modeling
- We have stronger predictive models for predicting adjacent words than words at arbitrary other locations and
- Causally structured: we believe that future events cannot influence the past

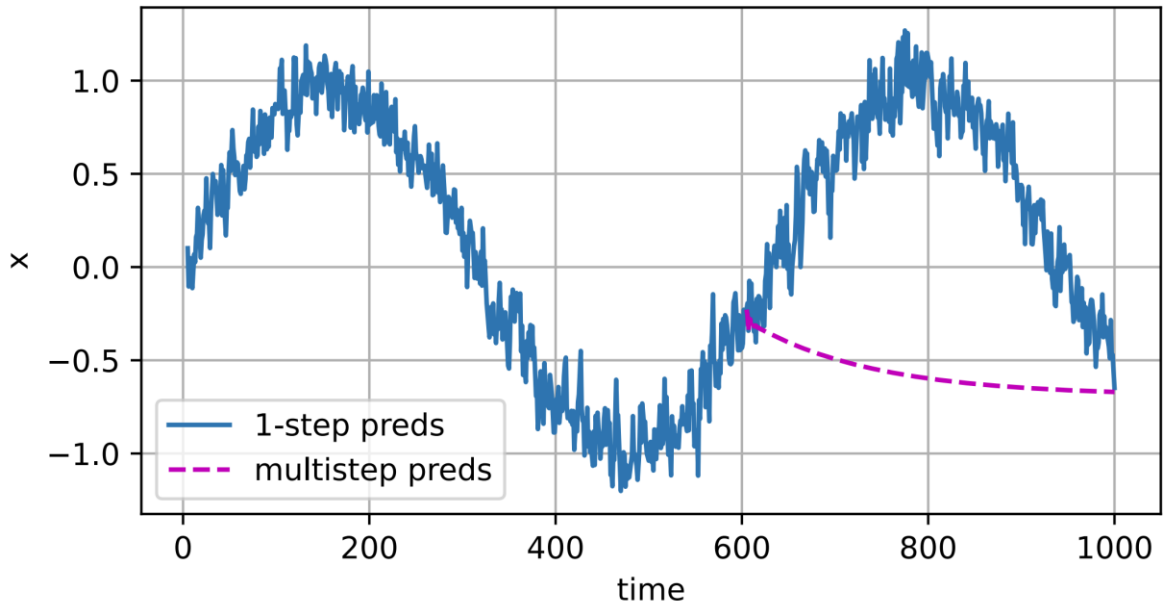
$$P(x_1, \ldots, x_T) = P(x_T) \prod_{t=T-1}^{1} P(x_t \mid x_{t+1}, \ldots, x_T).$$

PREDICTION

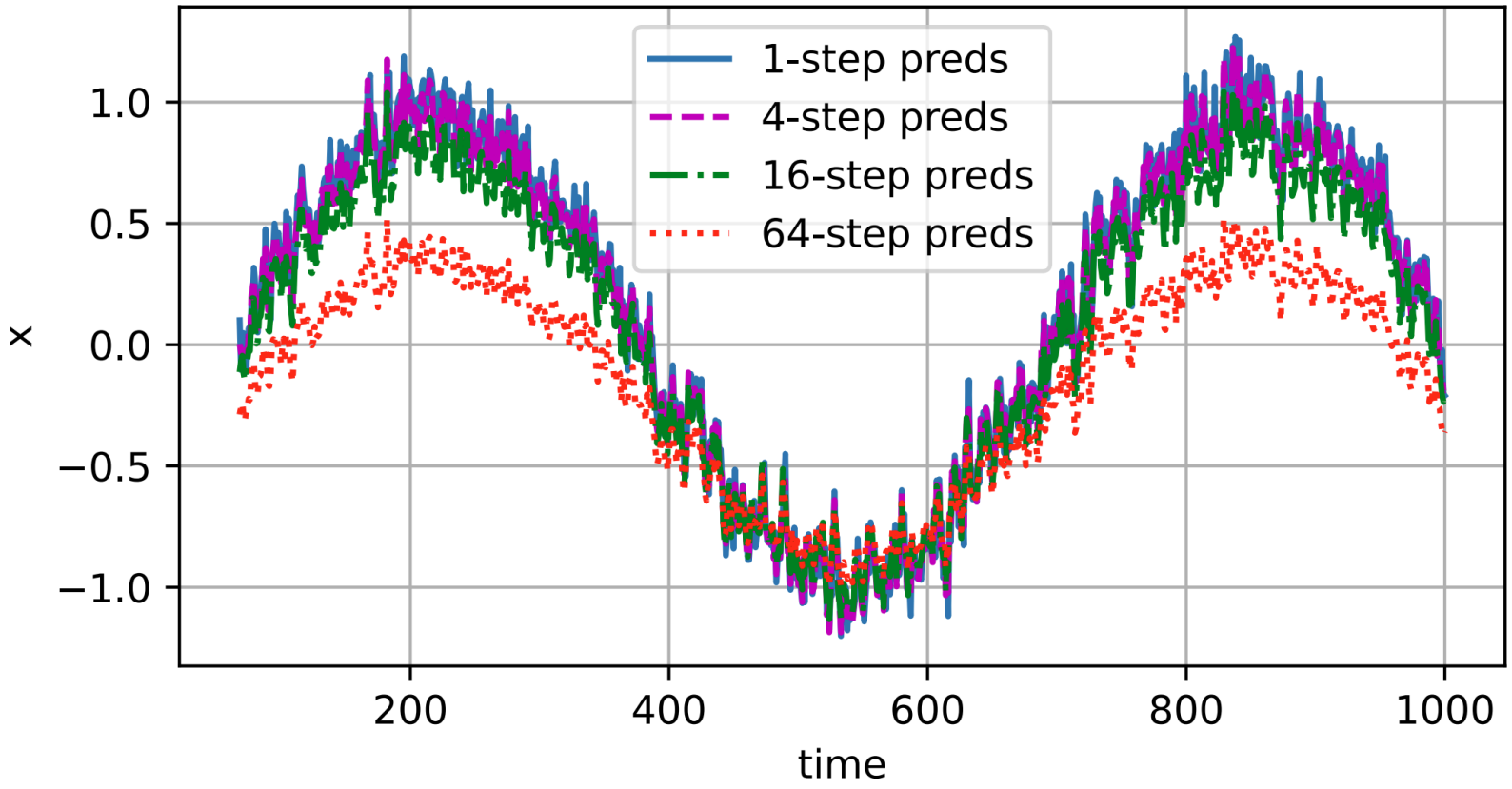

1.5
1.0
0.5
0.0
−0.5
−1.0
x
0
200
400
600
800
1000
time
labels
1-step preds

K-AHEAD PREDICTION

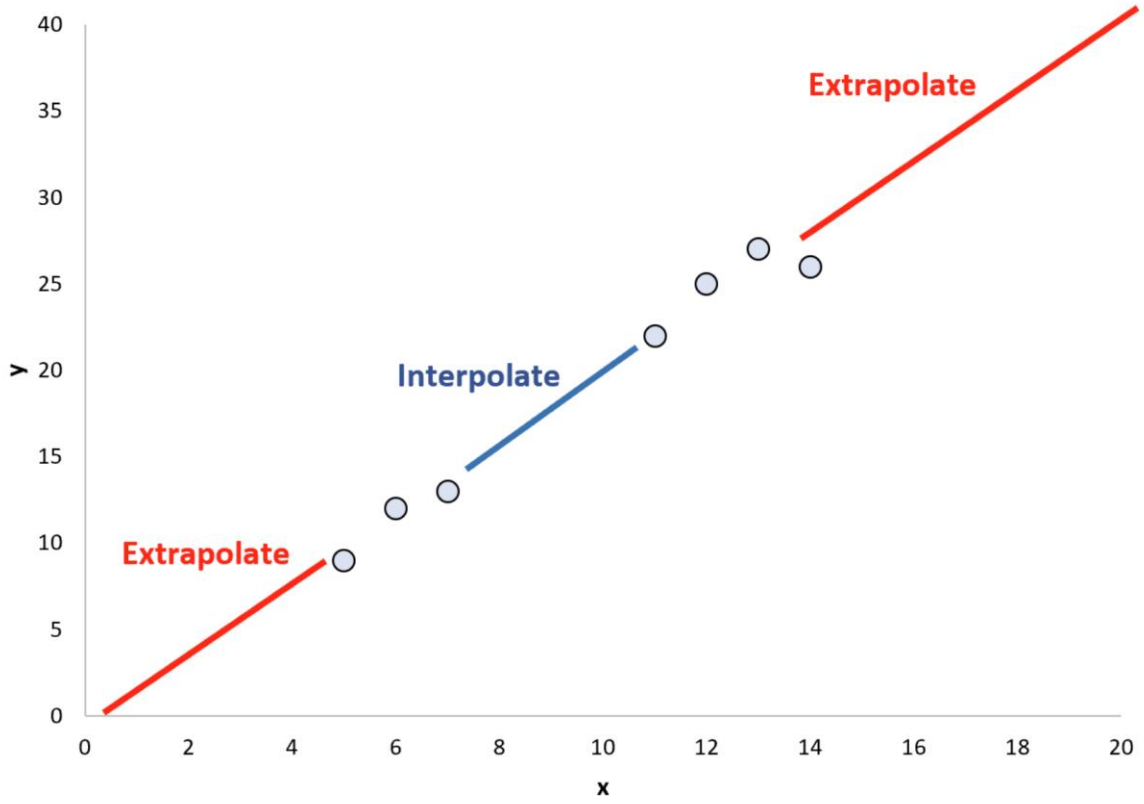
$$\hat{x}_{605} = f(x_{601}, x_{602}, x_{603}, x_{604}),$$
$$\hat{x}_{606} = f(x_{602}, x_{603}, x_{604}, \hat{x}_{605}),$$
$$\hat{x}_{607} = f(x_{603}, x_{604}, \hat{x}_{605}, \hat{x}_{606}),$$
$$\hat{x}_{608} = f(x_{604}, \hat{x}_{605}, \hat{x}_{606}, \hat{x}_{607}),$$
$$\hat{x}_{609} = f(\hat{x}_{605}, \hat{x}_{606}, \hat{x}_{607}, \hat{x}_{608}),$$
$$\vdots$$



K-AHEAD PREDICTION



INTERPOLATION VS EXTRAPOLATION



LANGUAGE MODELS

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}).$$

N-GRAMS

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2)P(x_3)P(x_4),$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_2)P(x_4 \mid x_3),$$

$$P(x_1, x_2, x_3, x_4) = P(x_1)P(x_2 \mid x_1)P(x_3 \mid x_1, x_2)P(x_4 \mid x_2, x_3).$$

WORD FREQUENCY

$$\hat{P}(\text{learning} \mid \text{deep}) = \frac{n(\text{deep, learning})}{n(\text{deep})},$$

LAPLACE SMOOTHING

$$\hat{P}(x) = \frac{n(x) + \epsilon_1/m}{n + \epsilon_1},$$

$$\hat{P}(x' \mid x) = \frac{n(x, x') + \epsilon_2 \hat{P}(x')}{n(x) + \epsilon_2},$$

$$\hat{P}(x'' \mid x, x') = \frac{n(x, x', x'') + \epsilon_3 \hat{P}(x'')}{n(x, x') + \epsilon_3}.$$

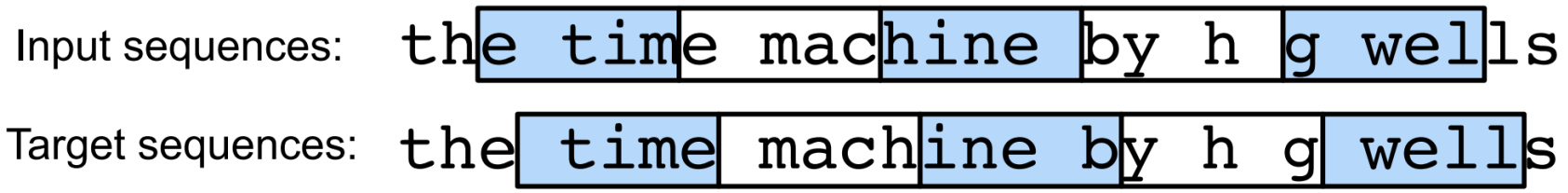
PERPLEXITY

1. “It is raining outside”
2. “It is raining banana tree”
3. “It is raining piouw;kcj pwepoiut”

$$\frac{1}{n}\sum_{t=1}^{n} -\log P(x_t \mid x_{t-1}, \ldots, x_1),$$

$$\exp\left(-\frac{1}{n}\sum_{t=1}^{n} \log P(x_t \mid x_{t-1}, \ldots, x_1)\right).$$

PARTITIONING SEQUENCES



2 - INPUTS TO RNNS

ONE-HOT ENCODING

Label Encoding

Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50

→

One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

torch.nn.functional.one_hot(*tensor*, *num_classes=-1*) → LongTensor

Takes LongTensor with index values of shape `(*)` and returns a tensor of shape `(*, num_classes)` that have zeros everywhere except where the index of last dimension matches the corresponding value of the input tensor, in which case it will be 1.

See also One-hot on Wikipedia .

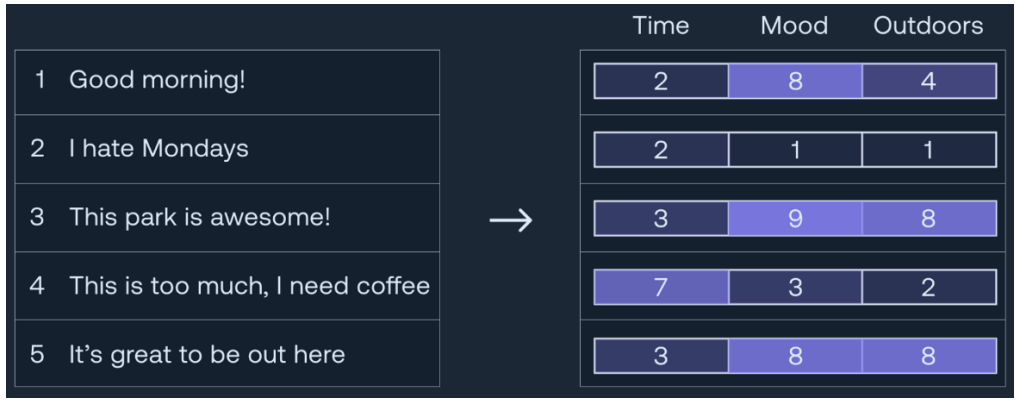
Parameters

- **tensor** (*LongTensor*) – class values of any shape.
- **num_classes** (*int*) – Total number of classes. If set to -1, the number of classes will be inferred as one greater than the largest class value in the input tensor.

Returns

LongTensor that has one more dimension with 1 values at the index of last dimension indicated by the input, and 0 everywhere else.

EMBEDDINGS



CLASS torch.nn.Embedding(*num_embeddings*, *embedding_dim*, *padding_idx=None*, *max_norm=None*, *norm_type=2.0*, *scale_grad_by_freq=False*, *sparse=False*, *_weight=None*, *_freeze=False*, *device=None*, *dtype=None*) [SOURCE]

A simple lookup table that stores embeddings of a fixed dictionary and size.

This module is often used to store word embeddings and retrieve them using indices. The input to the module is a list of indices, and the output is the corresponding word embeddings.

Parameters

- **num_embeddings** (*int*) – size of the dictionary of embeddings
- **embedding_dim** (*int*) – the size of each embedding vector
- **padding_idx** (*int*, *optional*) – If specified, the entries at `padding_idx` do not contribute to the gradient; therefore, the embedding vector at `padding_idx` is not updated during training, i.e. it remains as a fixed "pad". For a newly constructed Embedding, the embedding vector at `padding_idx` will default to all zeros, but can be updated to another value to be used as the padding vector.
- **max_norm** (*float*, *optional*) – If given, each embedding vector with norm larger than `max_norm` is renormalized to have norm `max_norm`.
- **norm_type** (*float*, *optional*) – The p of the p-norm to compute for the `max_norm` option. Default `2`.
- **scale_grad_by_freq** (*bool*, *optional*) – If given, this will scale gradients by the inverse of frequency of the words in the mini-batch. Default `False`.
- **sparse** (*bool*, *optional*) – If `True`, gradient w.r.t. `weight` matrix will be a sparse tensor. See Notes for more details regarding sparse gradients.

Variables

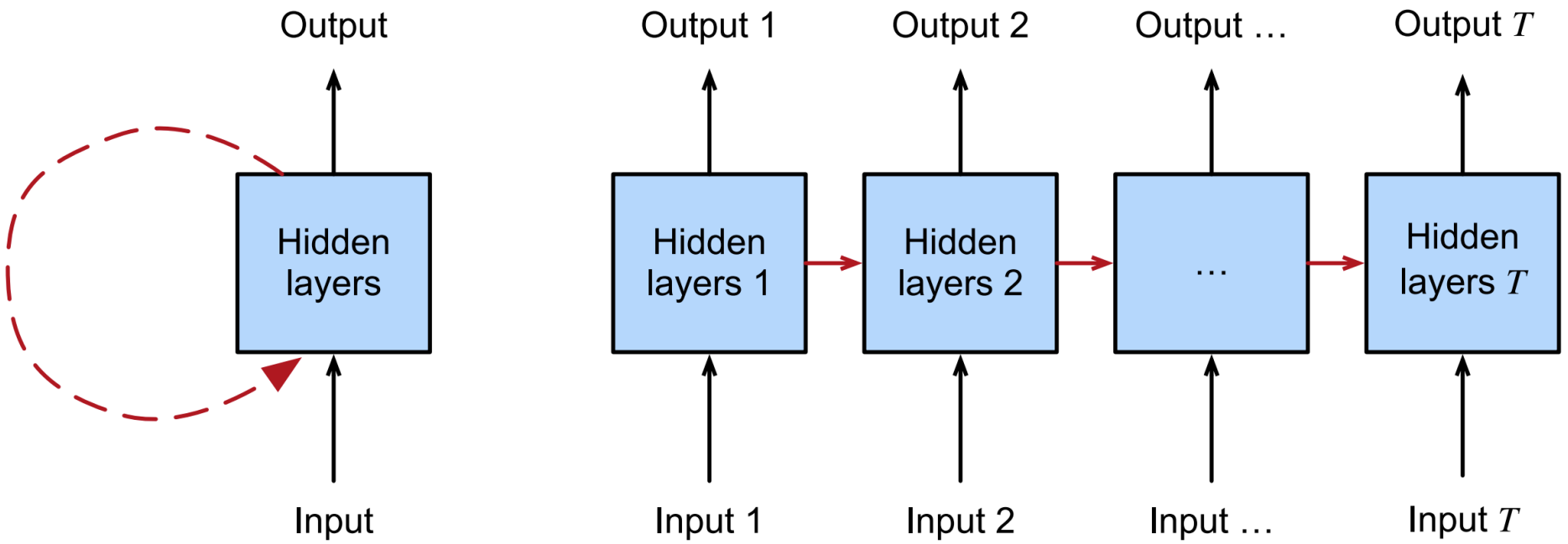
weight (*Tensor*) – the learnable weights of the module of shape (num_embeddings, embedding_dim) initialized from $\mathcal{N}(0, 1)$

Shape:

- Input: $(*)$, IntTensor or LongTensor of arbitrary shape containing the indices to extract
- Output: $(*, H)$, where $*$ is the input shape and $H = \text{embedding\_dim}$

3 – INTRODUCTION TO RNNS

RECURRENT NEURAL NETWORKS



RECURRENT NEURAL NETWORKS

$$P(x_t \mid x_{t-1}, \ldots, x_1) \approx P(x_t \mid h_{t-1}),$$

$$h_t = f(x_t, h_{t-1}).$$

NEURAL NETWORKS WITHOUT HIDDEN STATES

$$\mathbf{X} \in \mathbb{R}^{n \times d} \quad \mathbf{H} \in \mathbb{R}^{n \times h} \quad \mathbf{W}_{\text{xh}} \in \mathbb{R}^{d \times h} \quad \mathbf{O} \in \mathbb{R}^{n \times q} \quad \mathbf{b}_{\text{q}} \in \mathbb{R}^{1 \times q}$$

$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W}_{\text{xh}} + \mathbf{b}_{\text{h}}).$$

$$\mathbf{O} = \mathbf{H}\mathbf{W}_{\text{hq}} + \mathbf{b}_{\text{q}},$$

RNN WITH HIDDEN STATE

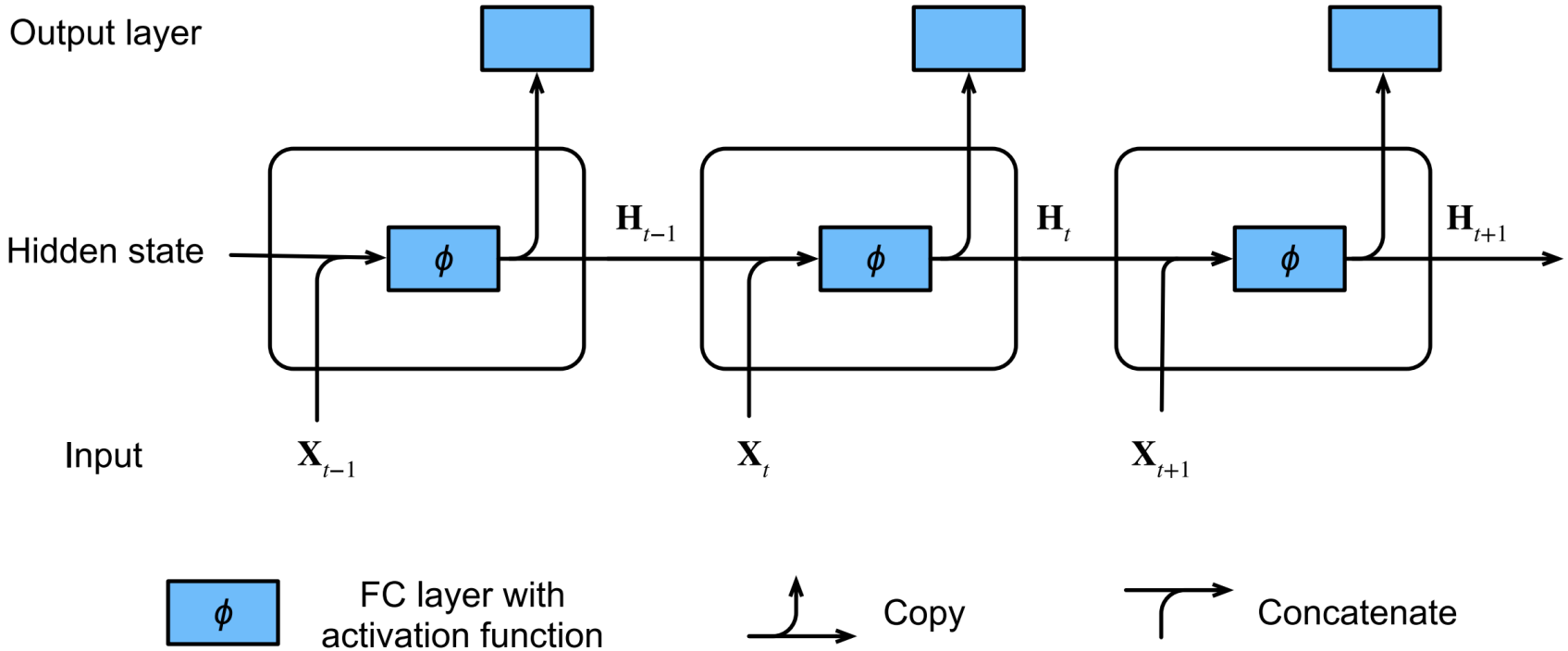
$$\mathbf{X}_t \in \mathbb{R}^{n \times d} \quad \mathbf{b}_q \in \mathbb{R}^{1 \times q} \quad \mathbf{H}_t \in \mathbb{R}^{n \times h}$$

$$\mathbf{W}_{hq} \in \mathbb{R}^{h \times q} \quad \mathbf{W}_{xh} \in \mathbb{R}^{d \times h} \quad \mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$$

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h).$$

$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q.$$

RNN WITH HIDDEN STATE



RNN IN PYTORCH

RNN

CLASS torch.nn.RNN(*self, input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0, bidirectional=False, device=None, dtype=None*) [SOURCE]

Apply a multi-layer Elman RNN with $\tanh$ or ReLU non-linearity to an input sequence. For each element in the input sequence, each layer computes the following function:

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

where h_t is the hidden state at time *t*, x_t is the input at time *t*, and $h_{(t-1)}$ is the hidden state of the previous layer at time *t-1* or the initial hidden state at time *0*. If `nonlinearity` is `'relu'`, then ReLU is used instead of $\tanh$.

```
input = torch.randn(64, 5, 10)
h0 = torch.randn(64, 2, 20)
model = nn.RNN(10, 20, 2, batch_first=True)
output, hn = rnn(input, h0)
```

Inputs: input, h_0

- **input**: tensor of shape (L, H_{in}) for unbatched input, (L, N, H_{in}) when `batch_first=False` or (N, L, H_{in}) when `batch_first=True` containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` or `torch.nn.utils.rnn.pack_sequence()` for details.
- **h_0**: tensor of shape $(D * \text{num\_layers}, H_{out})$ for unbatched input or $(D * \text{num\_layers}, N, H_{out})$ containing the initial hidden state for the input sequence batch. Defaults to zeros if not provided.

where:

$$\begin{aligned} N &= \text{batch size} \\ L &= \text{sequence length} \\ D &= 2 \text{ if bidirectional=True otherwise } 1 \\ H_{in} &= \text{input\_size} \\ H_{out} &= \text{hidden\_size} \end{aligned}$$

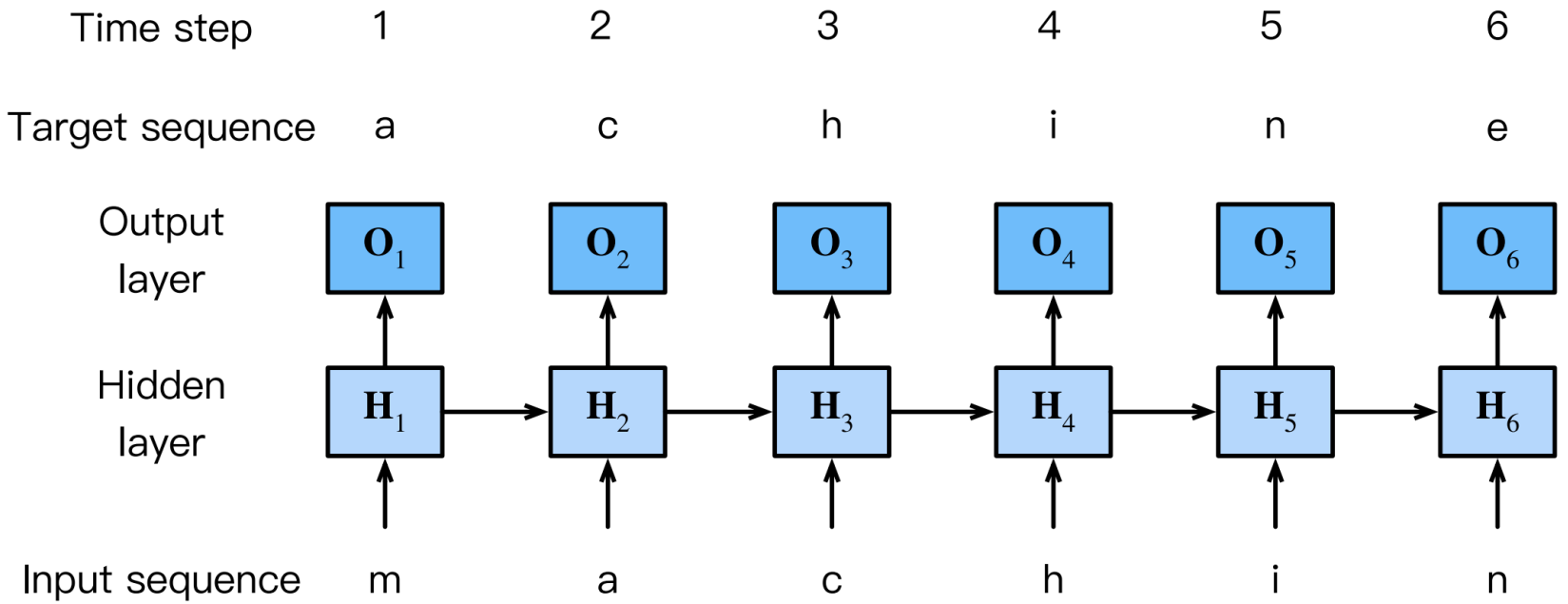
Outputs: output, h_n

- **output**: tensor of shape $(L, D * H_{out})$ for unbatched input, $(L, N, D * H_{out})$ when `batch_first=False` or $(N, L, D * H_{out})$ when `batch_first=True` containing the output features (*h_t*) from the last layer of the RNN, for each *t*. If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence.
- **h_n**: tensor of shape $(D * \text{num\_layers}, H_{out})$ for unbatched input or $(D * \text{num\_layers}, N, H_{out})$ containing the final hidden state for each element in the batch.

Variables

- **weight_ih_l[k]** – the learnable input-hidden weights of the k-th layer, of shape (*hidden_size, input_size*) for *k = 0*. Otherwise, the shape is (*hidden_size, num_directions * hidden_size*)
- **weight_hh_l[k]** – the learnable hidden-hidden weights of the k-th layer, of shape (*hidden_size, hidden_size*)
- **bias_ih_l[k]** – the learnable input-hidden bias of the k-th layer, of shape (*hidden_size*)
- **bias_hh_l[k]** – the learnable hidden-hidden bias of the k-th layer, of shape (*hidden_size*)

RNN CHARACTER LEVEL



GRADIENT CLIPPING

```
# compute outputs and loss
outputs = model(inputs)
loss_value = loss(outputs, targets)

# optimize
optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L\|\mathbf{x} - \mathbf{y}\|.$$

$$|f(\mathbf{x}) - f(\mathbf{x} - \eta\mathbf{g})| \leq L\eta\|\mathbf{g}\|.$$

$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right)\mathbf{g}.$$

4 - BACKPROP THROUGH TIME

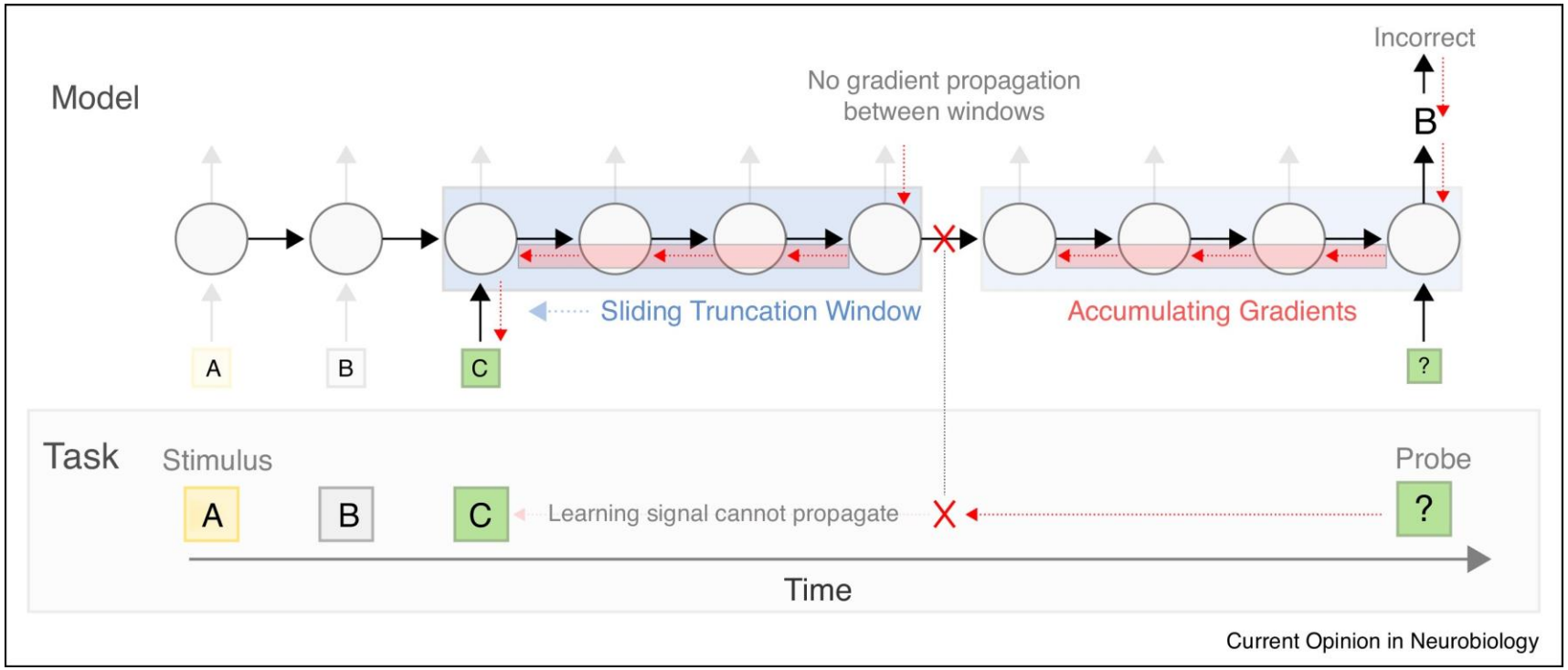
BACKPROP THROUGH TIME

$$h_t = f(x_t, h_{t-1}, w_\mathrm{h}),$$
$$o_t = g(h_t, w_\mathrm{o}),$$
$$L(x_1, \ldots, x_T, y_1, \ldots, y_T, w_\mathrm{h}, w_\mathrm{o}) = \frac{1}{T}\sum_{t=1}^{T} l(y_t, o_t).$$

$$\frac{\partial L}{\partial w_\mathrm{h}} = \frac{1}{T}\sum_{t=1}^{T} \frac{\partial l(y_t, o_t)}{\partial w_\mathrm{h}}$$
$$= \frac{1}{T}\sum_{t=1}^{T} \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_\mathrm{o})}{\partial h_t} \frac{\partial h_t}{\partial w_\mathrm{h}}.$$

$$\frac{\partial h_t}{\partial w_\mathrm{h}} = \frac{\partial f(x_t, h_{t-1}, w_\mathrm{h})}{\partial w_\mathrm{h}} + \frac{\partial f(x_t, h_{t-1}, w_\mathrm{h})}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_\mathrm{h}}.$$

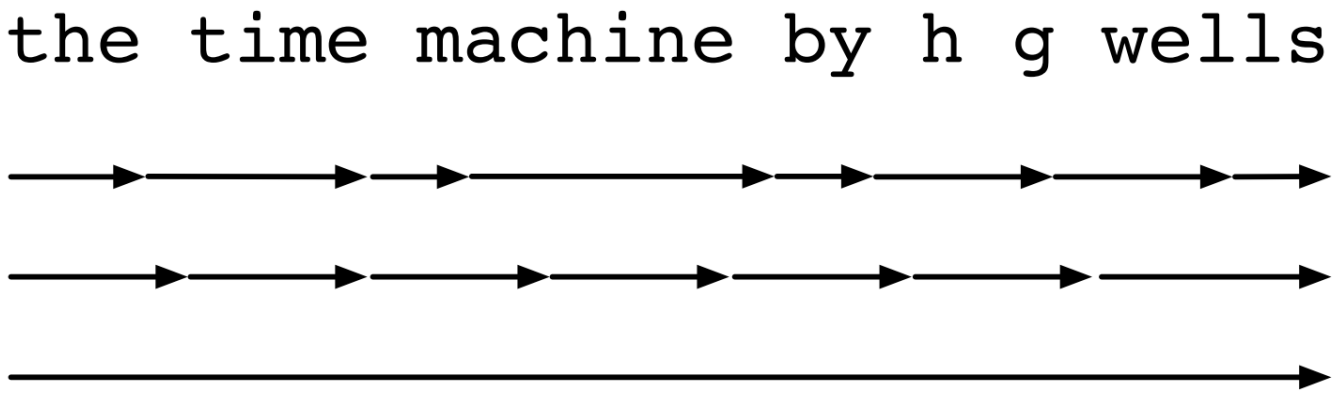
FULL COMPUTATION VS TRUNCATED COMPUTATION



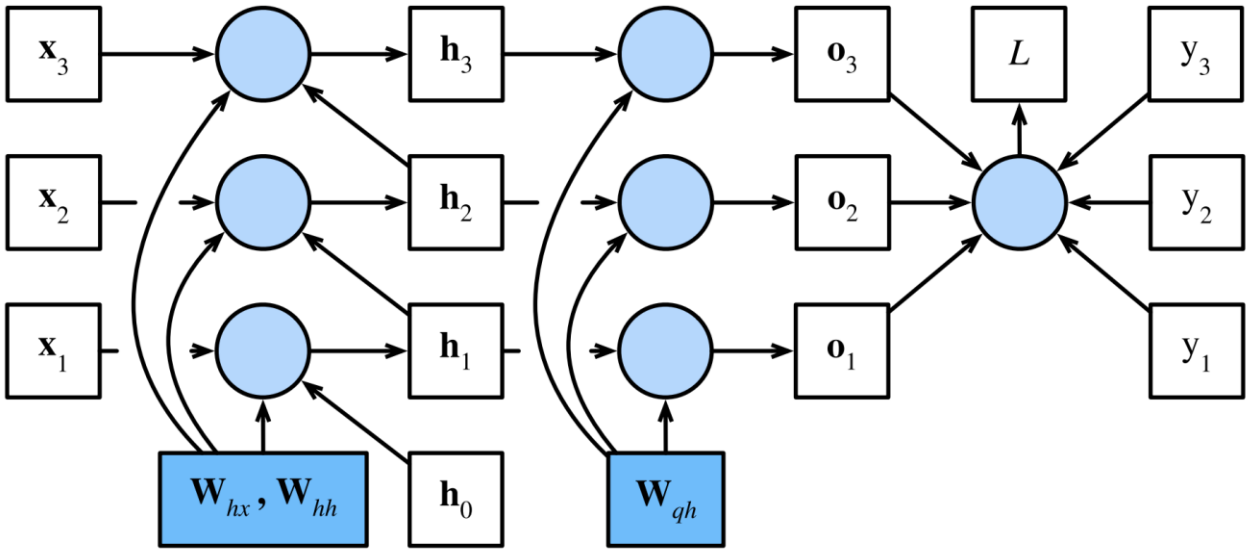
RANDOMIZED TRUNCATION

$$z_t = \frac{\partial f(x_t, h_{t-1}, w_\mathrm{h})}{\partial w_\mathrm{h}} + \xi_t \frac{\partial f(x_t, h_{t-1}, w_\mathrm{h})}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_\mathrm{h}}.$$

COMPARING STRATEGIES



BACKPROP THROUGH TIME IN DETAIL



$$\mathbf{h}_t = \mathbf{W}_{\text{hx}}\mathbf{x}_t + \mathbf{W}_{\text{hh}}\mathbf{h}_{t-1},$$

$$\mathbf{o}_t = \mathbf{W}_{\text{qh}}\mathbf{h}_t,$$

$$L = \frac{1}{T}\sum_{t=1}^{T} l(\mathbf{o}_t, y_t).$$

BACKPROP THROUGH TIME IN DETAIL

$$\frac{\partial L}{\partial \mathbf{o}_t} = \frac{\partial l(\mathbf{o}_t, y_t)}{T \cdot \partial \mathbf{o}_t} \in \mathbb{R}^q.$$

$$\frac{\partial L}{\partial \mathbf{W}_{\text{qh}}} = \sum_{t=1}^{T} \text{prod}\left(\frac{\partial L}{\partial \mathbf{o}_t}, \frac{\partial \mathbf{o}_t}{\partial \mathbf{W}_{\text{qh}}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \mathbf{o}_t} \mathbf{h}_t^\top,$$

BACKPROP THROUGH TIME IN DETAIL

$$\frac{\partial L}{\partial \mathbf{h}_T} = \text{prod}\left(\frac{\partial L}{\partial \mathbf{o}_T}, \frac{\partial \mathbf{o}_T}{\partial \mathbf{h}_T}\right) = \mathbf{W}_{\text{qh}}^\top \frac{\partial L}{\partial \mathbf{o}_T}.$$

$$\frac{\partial L}{\partial \mathbf{h}_t} = \text{prod}\left(\frac{\partial L}{\partial \mathbf{h}_{t+1}}, \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}\right) + \text{prod}\left(\frac{\partial L}{\partial \mathbf{o}_t}, \frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_t}\right) = \mathbf{W}_{\text{hh}}^\top \frac{\partial L}{\partial \mathbf{h}_{t+1}} + \mathbf{W}_{\text{qh}}^\top \frac{\partial L}{\partial \mathbf{o}_t}.$$

$$\frac{\partial L}{\partial \mathbf{h}_t} = \sum_{i=t}^{T} \left(\mathbf{W}_{\text{hh}}^\top\right)^{T-i} \mathbf{W}_{\text{qh}}^\top \frac{\partial L}{\partial \mathbf{o}_{T+t-i}}.$$

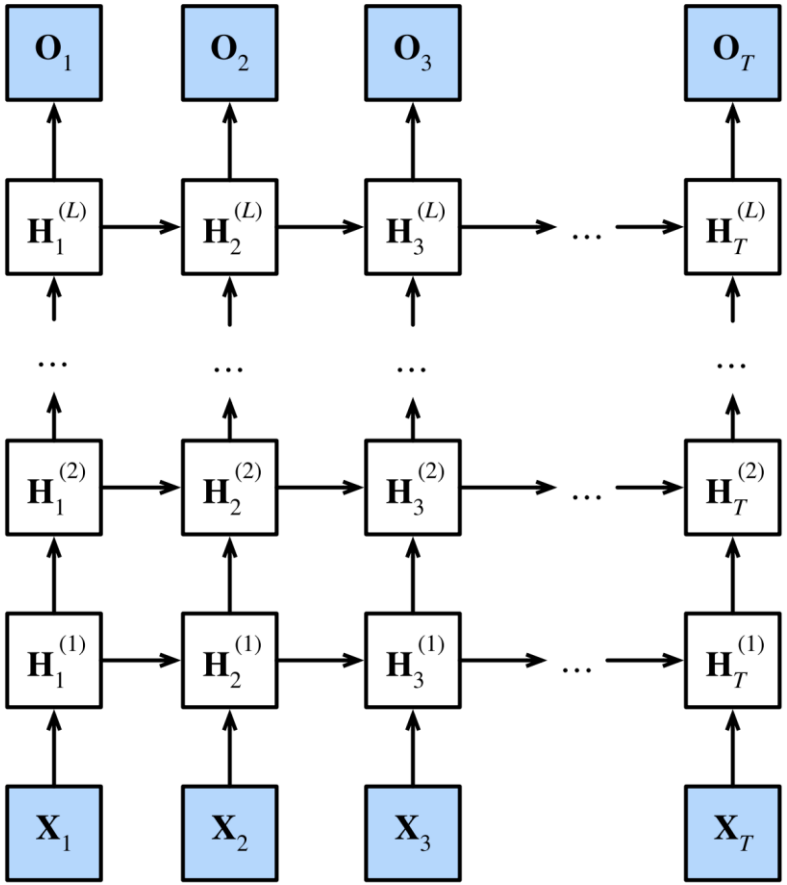
BACKPROP THROUGH TIME IN DETAIL

$$\frac{\partial L}{\partial \mathbf{W}_{\text{hx}}} = \sum_{t=1}^{T} \text{prod}\left(\frac{\partial L}{\partial \mathbf{h}_t}, \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{\text{hx}}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \mathbf{h}_t} \mathbf{x}_t^\top,$$

$$\frac{\partial L}{\partial \mathbf{W}_{\text{hh}}} = \sum_{t=1}^{T} \text{prod}\left(\frac{\partial L}{\partial \mathbf{h}_t}, \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{\text{hh}}}\right) = \sum_{t=1}^{T} \frac{\partial L}{\partial \mathbf{h}_t} \mathbf{h}_{t-1}^\top,$$

5 - DEEP RNNS

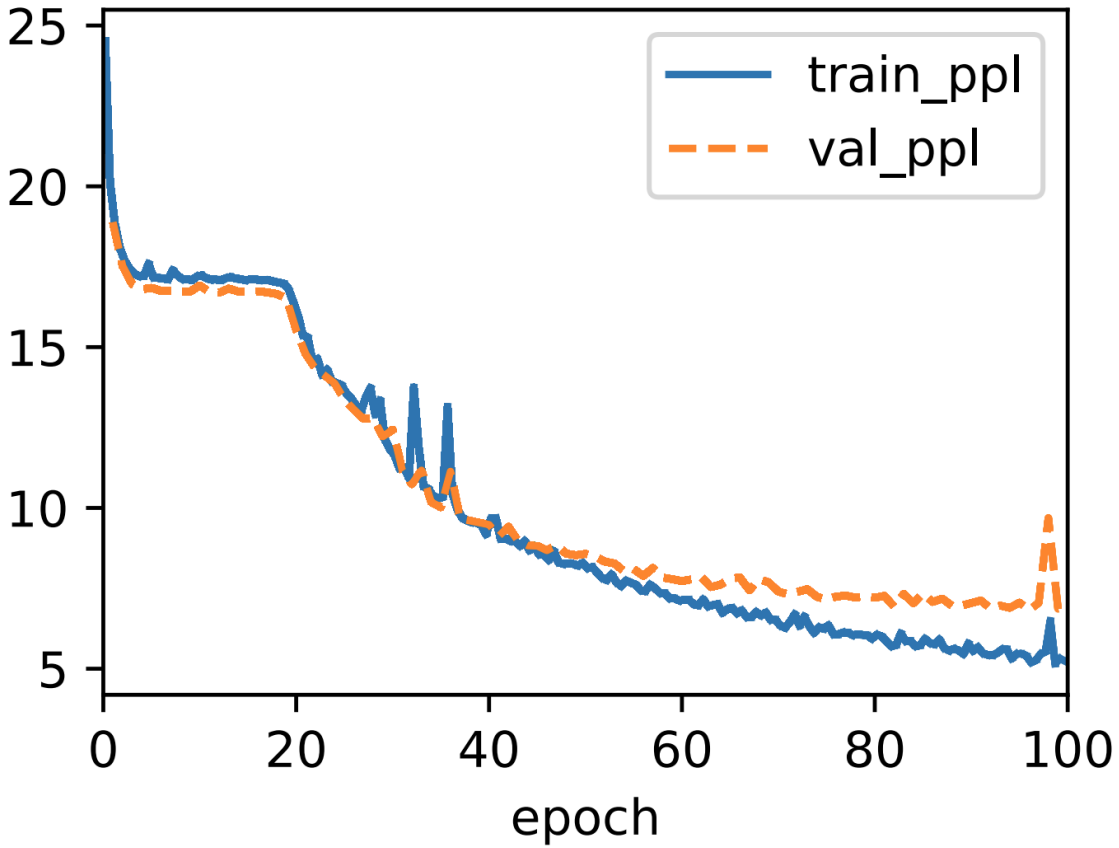
INTRODUCTION TO DEEP RNNS



MATHEMATICAL FORMULATION

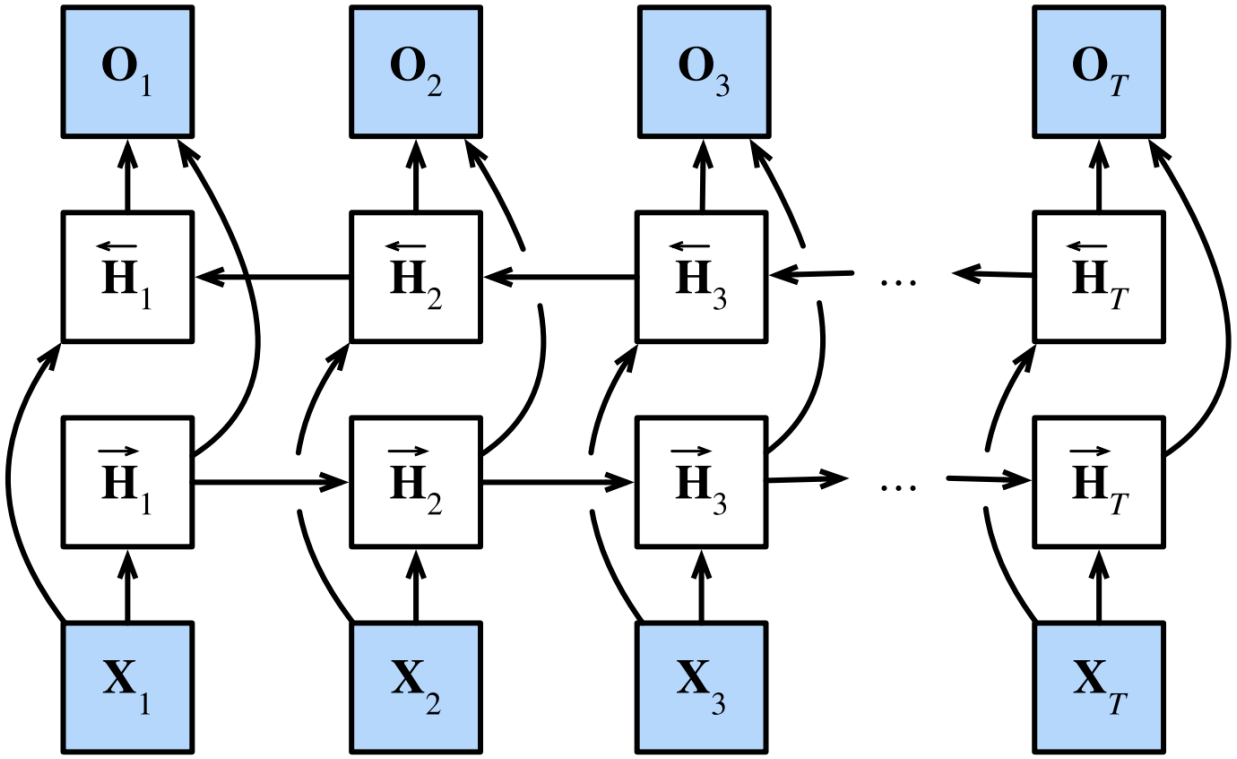
$$\mathbf{H}_t^{(l)} = \phi_l(\mathbf{H}_t^{(l-1)}\mathbf{W}_{\mathrm{xh}}^{(l)} + \mathbf{H}_{t-1}^{(l)}\mathbf{W}_{\mathrm{hh}}^{(l)} + \mathbf{b}_{\mathrm{h}}^{(l)}),$$

$$\mathbf{O}_t = \mathbf{H}_t^{(L)}\mathbf{W}_{\mathrm{hq}} + \mathbf{b}_{\mathrm{q}},$$


25
20
15
10
5
0
20
40
60
80
100
epoch
train_ppl
val_ppl

6 - BIDIRECTIONAL RNNS

ARCHITECTURE



MATHEMATICAL FORMULATION

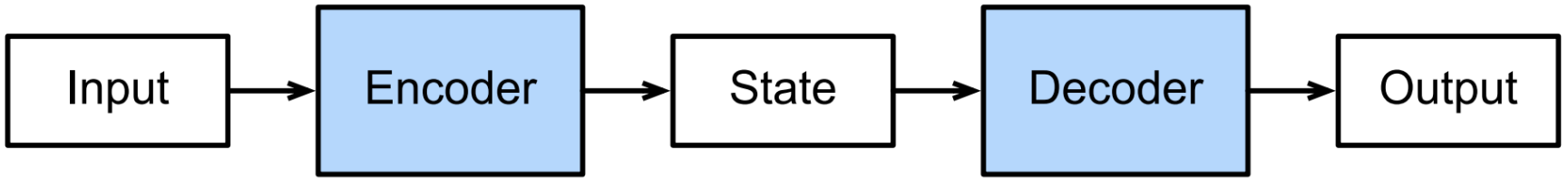
$$\overrightarrow{\mathbf{H}}_t = \phi(\mathbf{X}_t \mathbf{W}_{\text{xh}}^{(f)} + \overrightarrow{\mathbf{H}}_{t-1} \mathbf{W}_{\text{hh}}^{(f)} + \mathbf{b}_{\text{h}}^{(f)}),$$

$$\overleftarrow{\mathbf{H}}_t = \phi(\mathbf{X}_t \mathbf{W}_{\text{xh}}^{(b)} + \overleftarrow{\mathbf{H}}_{t+1} \mathbf{W}_{\text{hh}}^{(b)} + \mathbf{b}_{\text{h}}^{(b)}),$$

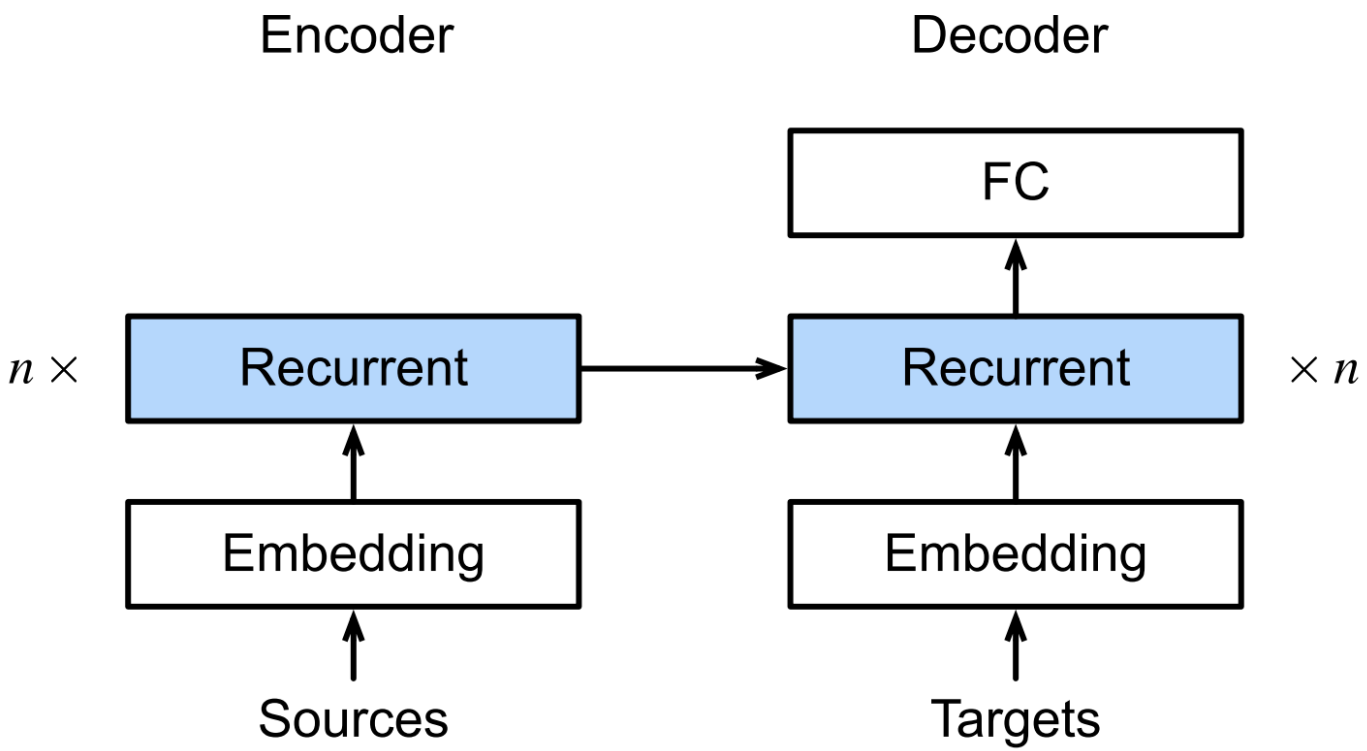
$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{\text{hq}} + \mathbf{b}_{\text{q}}.$$

7 - ENCODER & DECODER

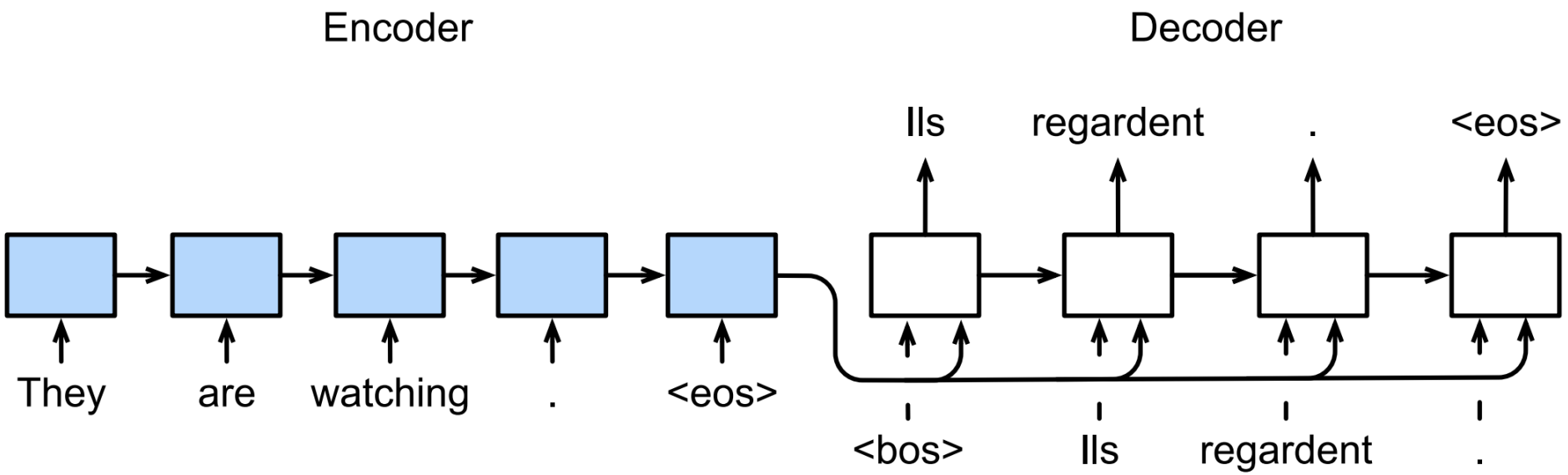
ARCHITECTURE



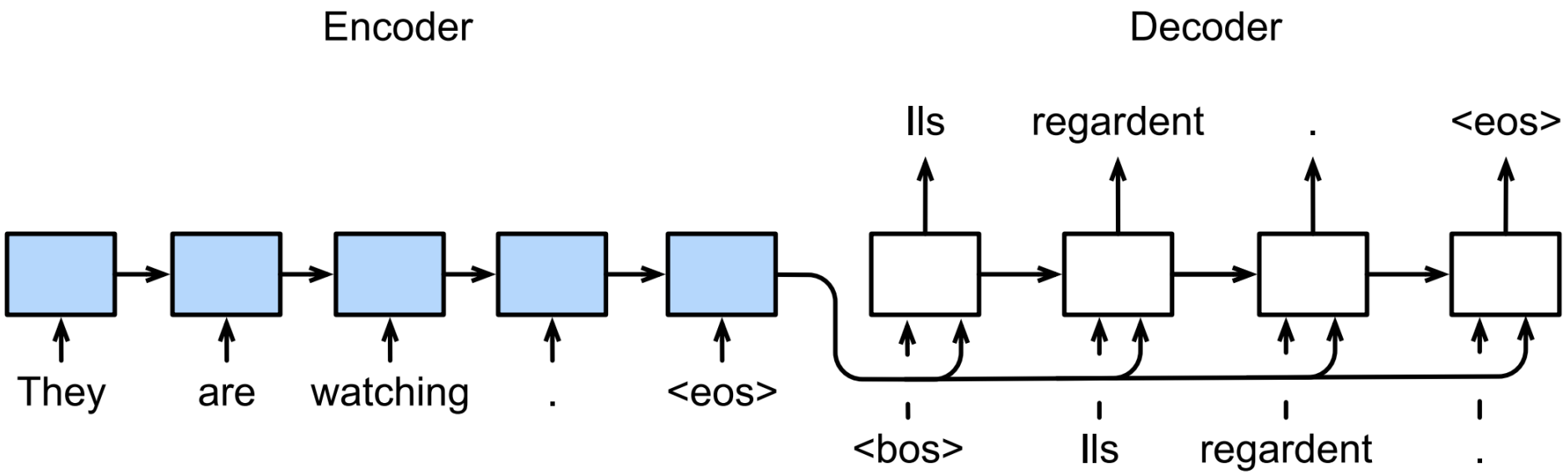
SEQUENCE TO SEQUENCE



SEQUENCE TO SEQUENCE TRANSLATION



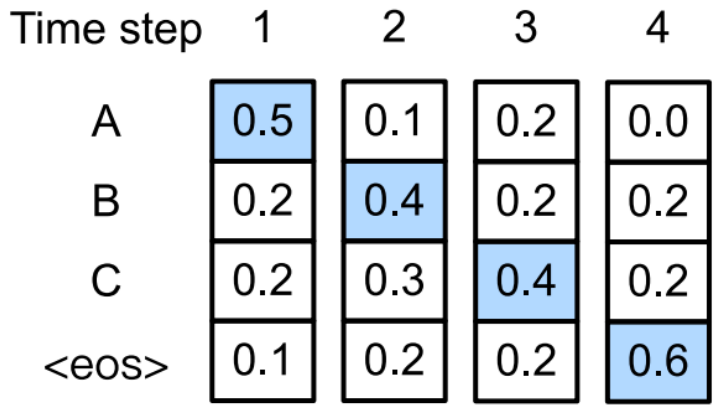
TRANSLATION PREDICTION



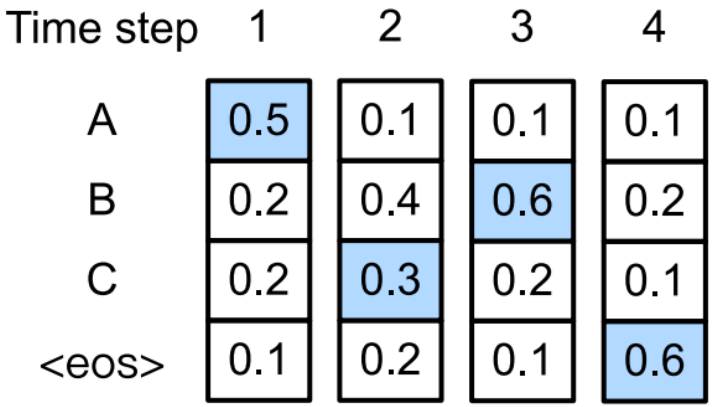
8 - SEARCH

GREEDY SEARCH

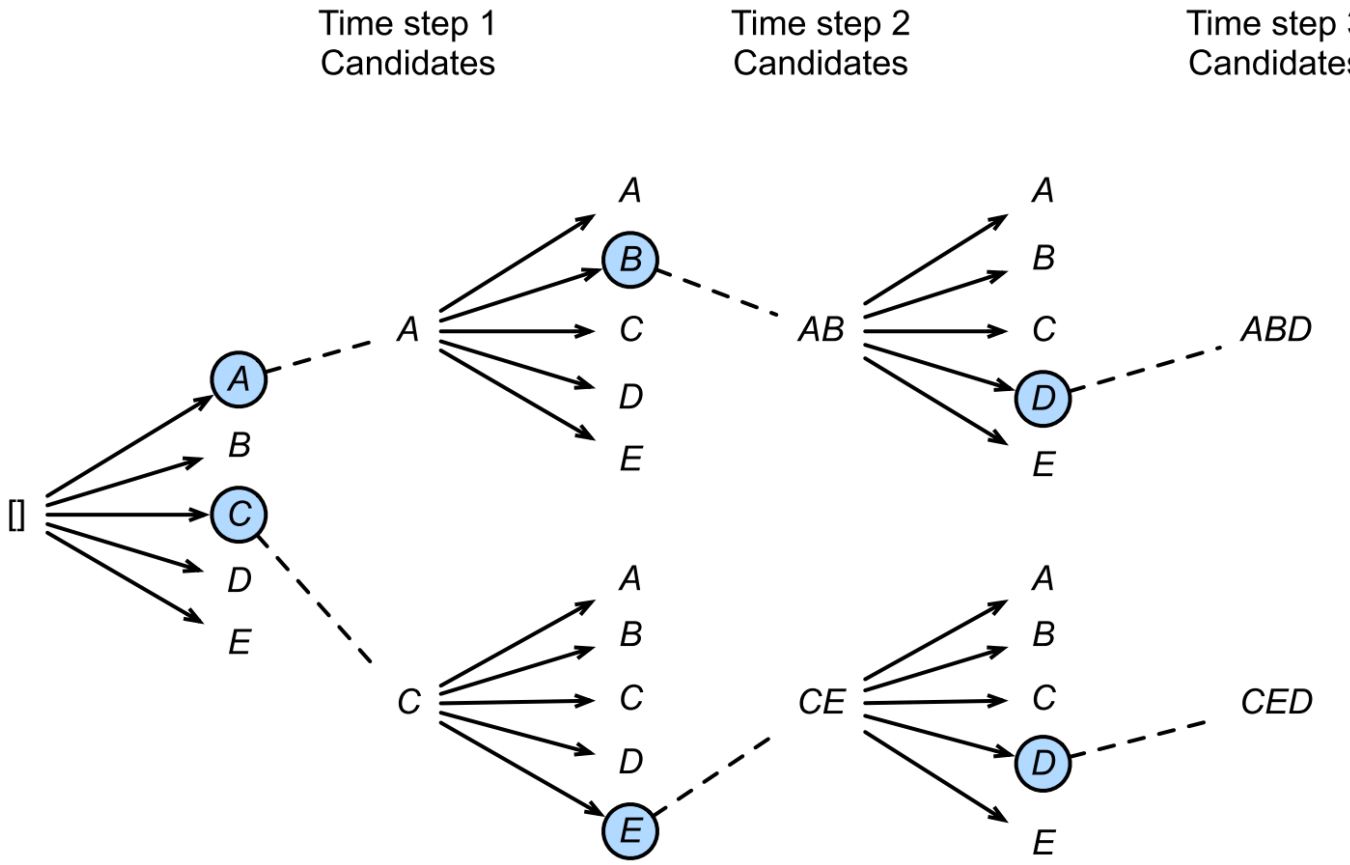
Greedy search



Most likely sentence (cond probs. change!)

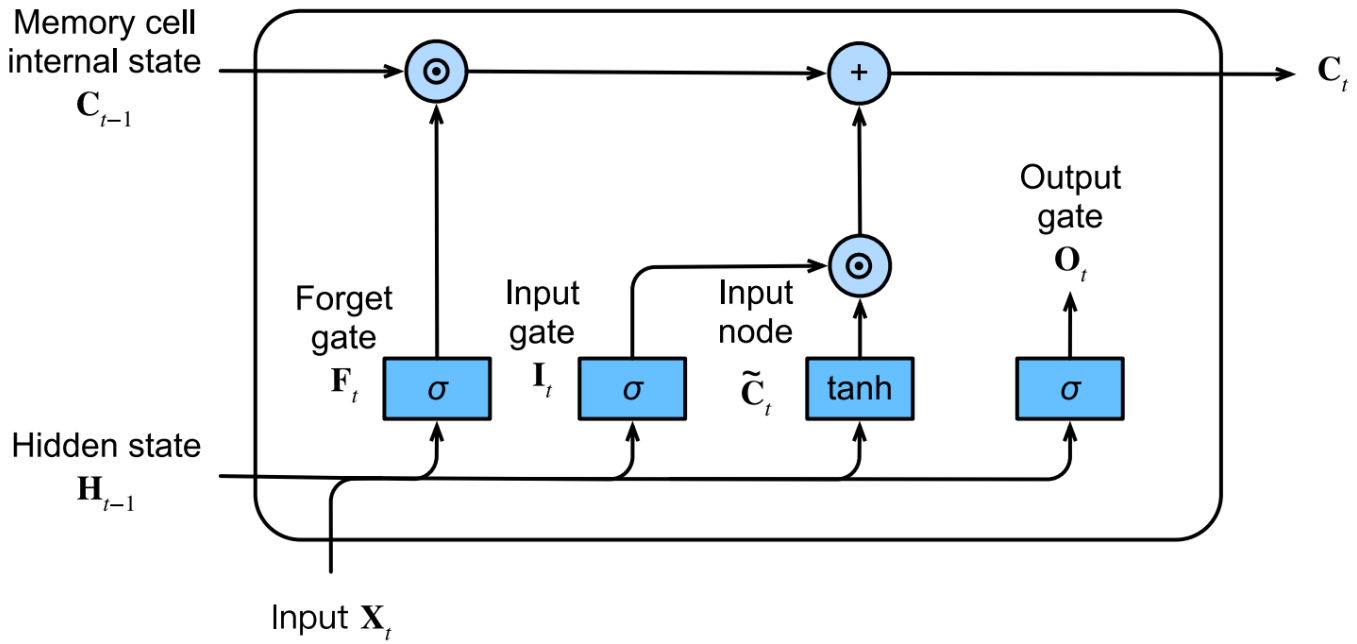


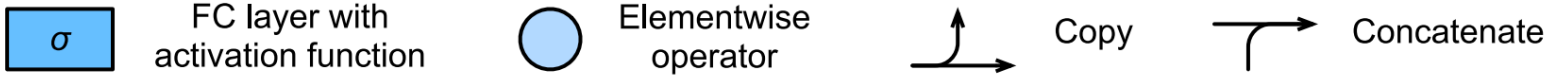
BEAM SEARCH


Time step 1
Candidates
Time step 2
Candidates
Time step 3
Candidates
[]
A
B
C
D
E
A
B
C
D
E
AB
ABD
C
CE
CED

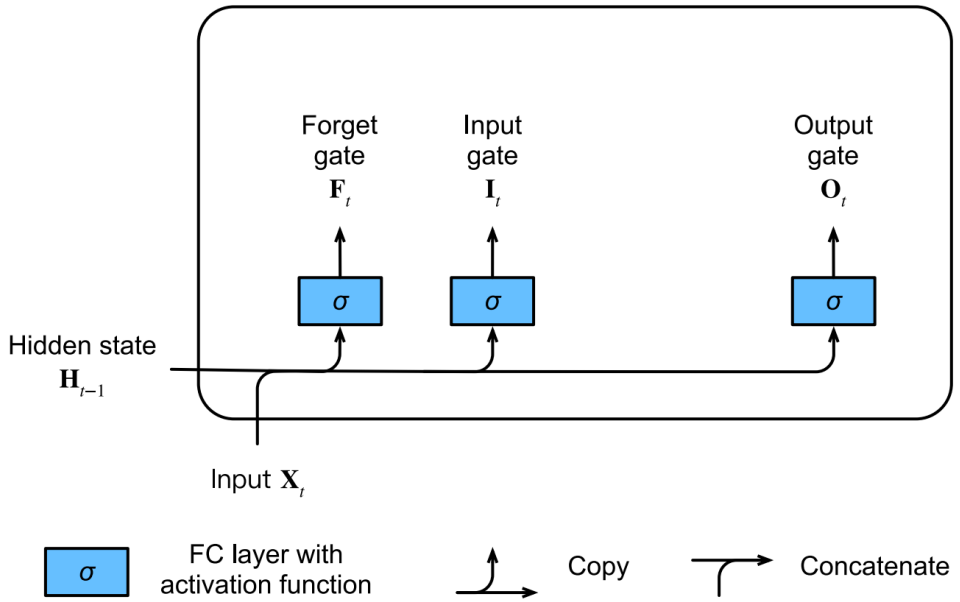
9 - MODERN RNNS

LSTM


Memory cell
internal state
$\mathbf{C}_{t-1}$
$\mathbf{C}_t$
Forget
gate
$\mathbf{F}_t$
Input
gate
$\mathbf{I}_t$
Input
node
$\tilde{\mathbf{C}}_t$
Output
gate
$\mathbf{O}_t$
σ
σ
tanh
σ
Hidden state
$\mathbf{H}_{t-1}$
Input $\mathbf{X}_t$


σ
FC layer with
activation function
Elementwise
operator
Copy
Concatenate

INPUT GATE, FORGET GATE, AND OUTPUT GATE



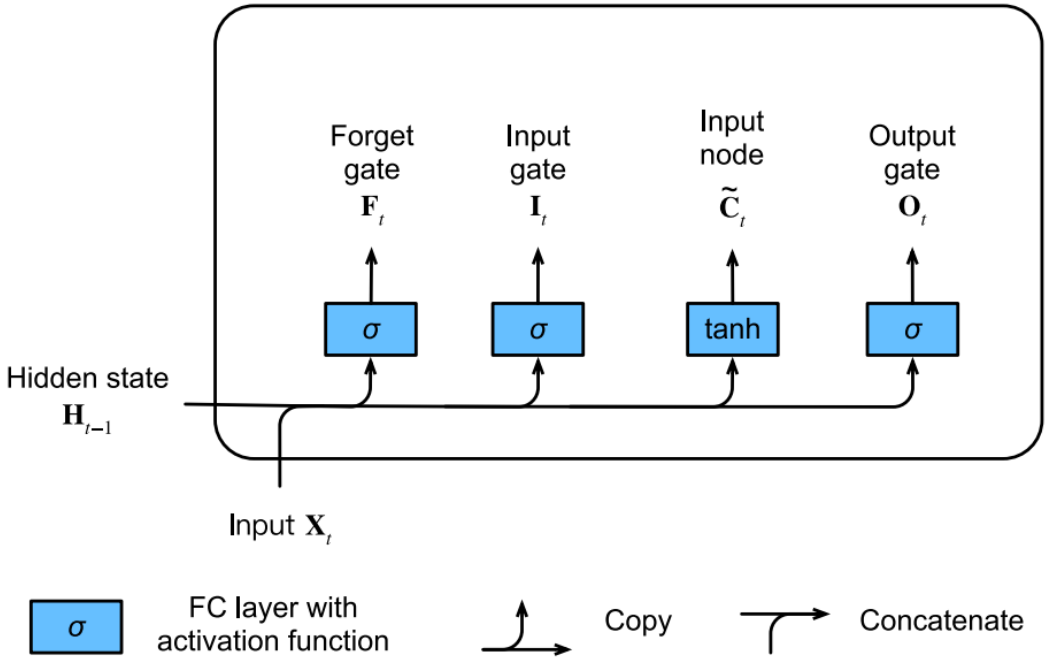
$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_{\text{xi}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hi}} + \mathbf{b}_{\text{i}}),$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_{\text{xf}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hf}} + \mathbf{b}_{\text{f}}),$$

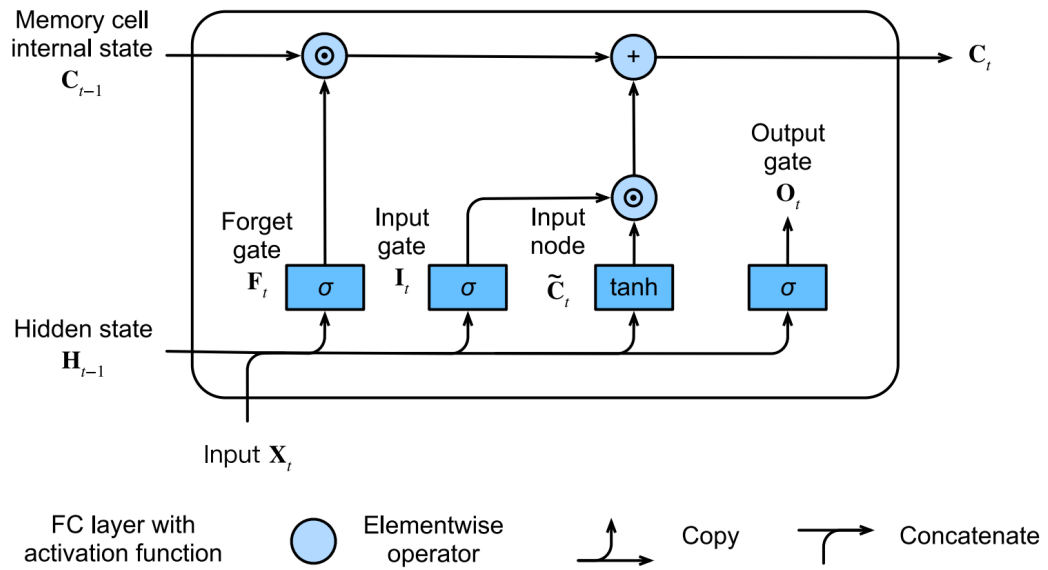
$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_{\text{xo}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{ho}} + \mathbf{b}_{\text{o}}),$$

INPUT NODE

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{\text{xc}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hc}} + \mathbf{b}_{\text{c}}),$$



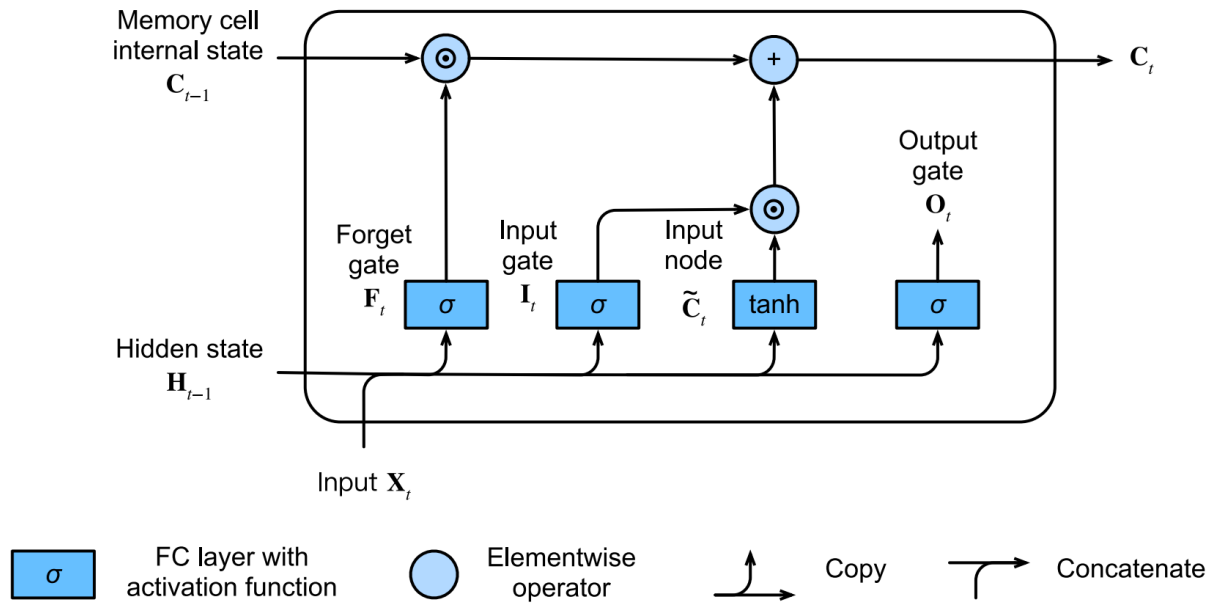
MEMORY CELL INTERNAL STATE



$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$$

HIDDEN STATE

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t).$$



LSTM PYTORCH

```python
rnn = nn.LSTM(10, 20, 2)
input = torch.randn(5, 3, 10)
h0 = torch.randn(2, 3, 20)
c0 = torch.randn(2, 3, 20)
output, (hn, cn) = rnn(input, (h0, c0))
```

Inputs: input, (h_0, c_0)

- **input**: tensor of shape (L, H_{in}) for unbatched input, (L, N, H_{in}) when `batch_first=False` or (N, L, H_{in}) when `batch_first=True` containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` or `torch.nn.utils.rnn.pack_sequence()` for details.
- **h_0**: tensor of shape $(D * \text{num\_layers}, H_{out})$ for unbatched input or $(D * \text{num\_layers}, N, H_{out})$ containing the initial hidden state for each element in the input sequence. Defaults to zeros if (h_0, c_0) is not provided.
- **c_0**: tensor of shape $(D * \text{num\_layers}, H_{cell})$ for unbatched input or $(D * \text{num\_layers}, N, H_{cell})$ containing the initial cell state for each element in the input sequence. Defaults to zeros if (h_0, c_0) is not provided.

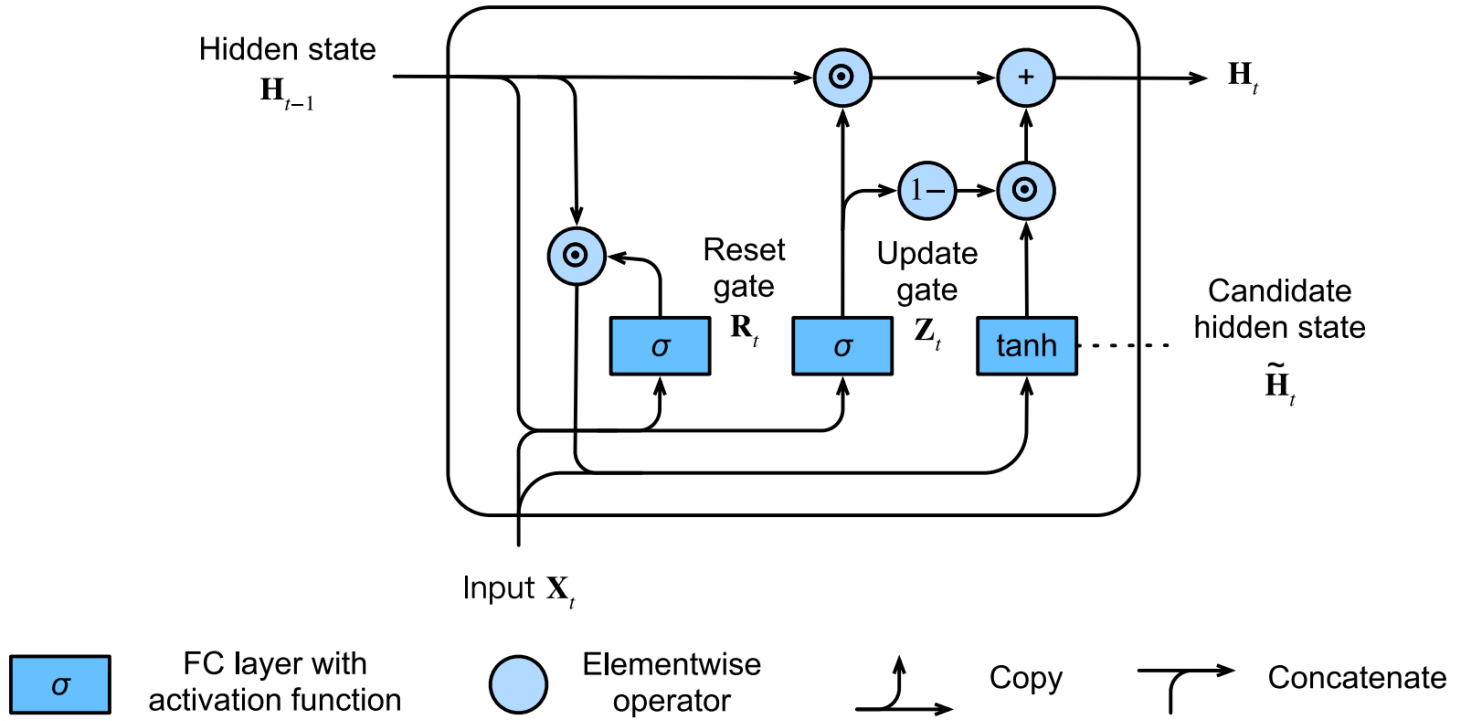
where:

$$
\begin{aligned}
N &= \text{batch size} \\
L &= \text{sequence length} \\
D &= 2 \text{ if bidirectional=True otherwise } 1 \\
H_{in} &= \text{input\_size} \\
H_{cell} &= \text{hidden\_size} \\
H_{out} &= \text{proj\_size if proj\_size} > 0 \text{ otherwise hidden\_size}
\end{aligned}
$$

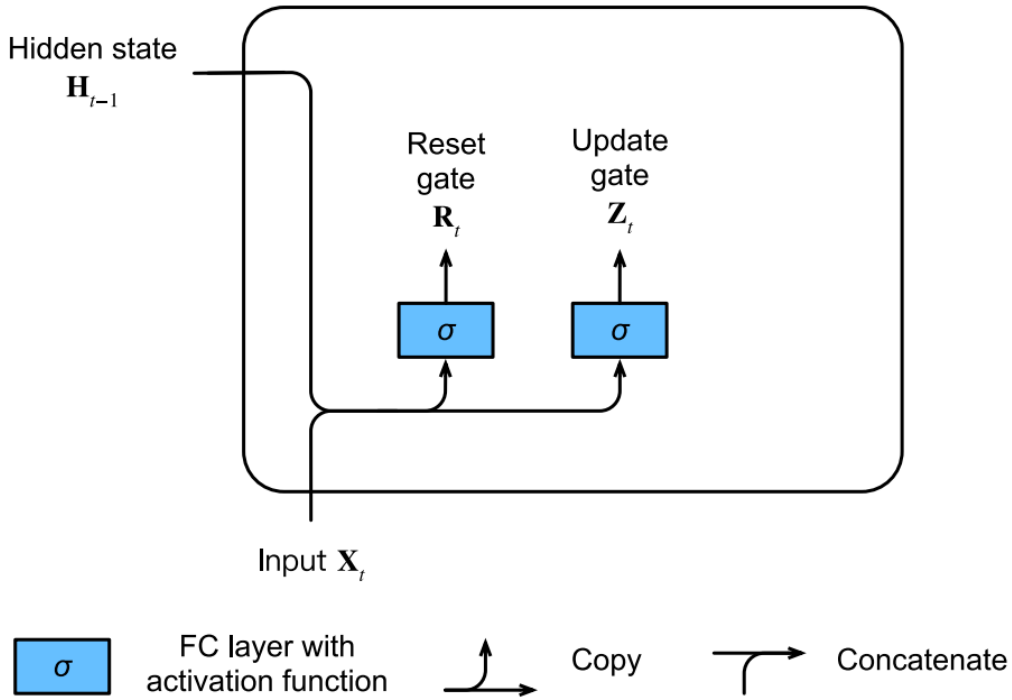
Outputs: output, (h_n, c_n)

- **output**: tensor of shape $(L, D * H_{out})$ for unbatched input, $(L, N, D * H_{out})$ when `batch_first=False` or $(N, L, D * H_{out})$ when `batch_first=True` containing the output features (*h_t*) from the last layer of the LSTM, for each *t*. If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence. When `bidirectional=True`, *output* will contain a concatenation of the forward and reverse hidden states at each time step in the sequence.
- **h_n**: tensor of shape $(D * \text{num\_layers}, H_{out})$ for unbatched input or $(D * \text{num\_layers}, N, H_{out})$ containing the final hidden state for each element in the sequence. When `bidirectional=True`, *h_n* will contain a concatenation of the final forward and reverse hidden states, respectively.
- **c_n**: tensor of shape $(D * \text{num\_layers}, H_{cell})$ for unbatched input or $(D * \text{num\_layers}, N, H_{cell})$ containing the final cell state for each element in the sequence. When `bidirectional=True`, *c_n* will contain a concatenation of the final forward and reverse cell states, respectively.

GRU


Hidden state
$\mathbf{H}_{t-1}$
$\mathbf{H}_t$
+
1−
Reset gate
$\mathbf{R}_t$
Update gate
$\mathbf{Z}_t$
Candidate hidden state
$\tilde{\mathbf{H}}_t$
σ
σ
tanh
Input $\mathbf{X}_t$
σ
FC layer with activation function
Elementwise operator
Copy
Concatenate

RESET GATE AND FORGET GATE

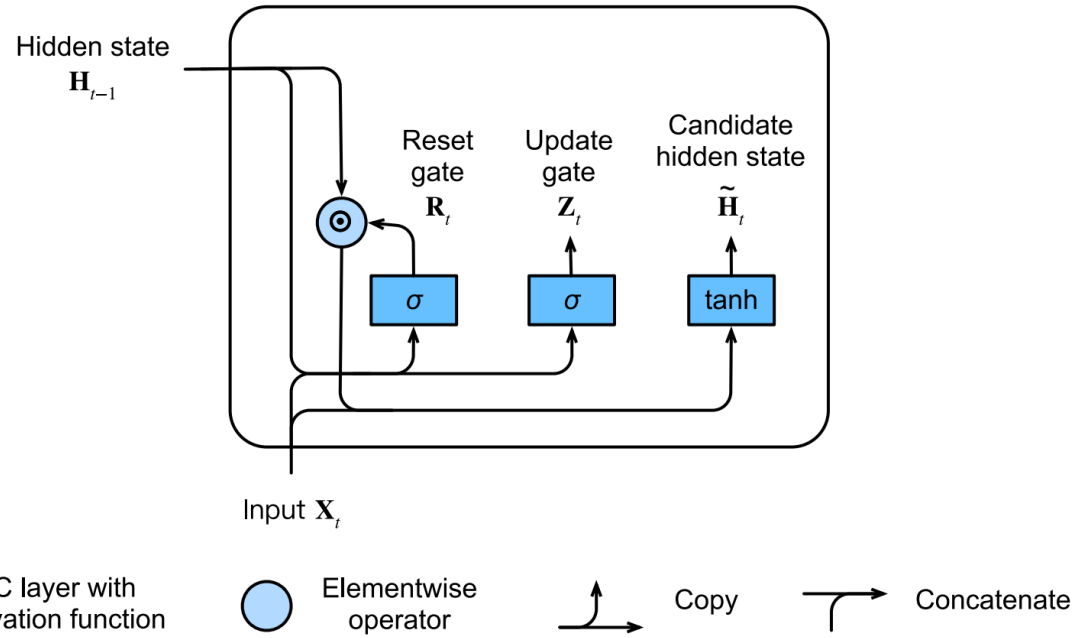


$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r),$$

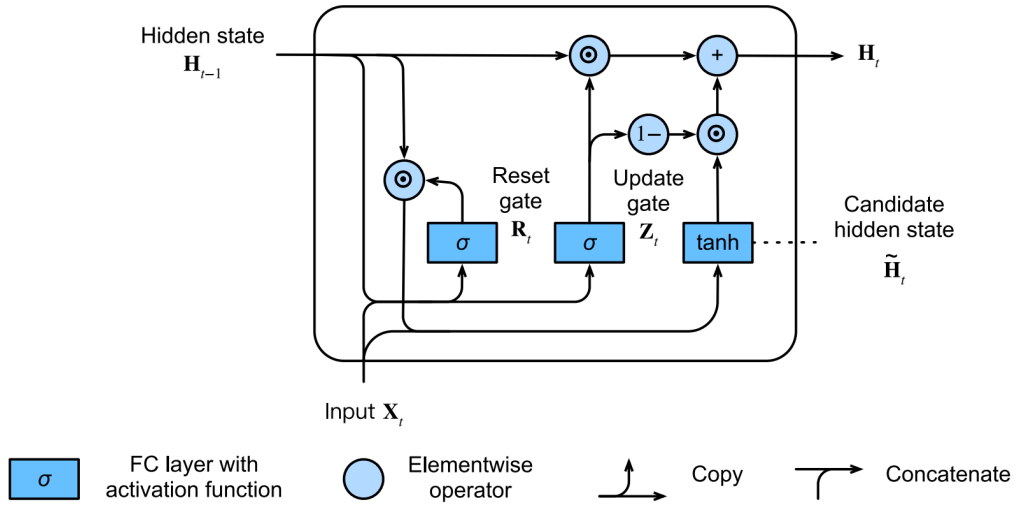
$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z),$$

CANDIDATE HIDDEN STATE

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{\text{xh}} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{\text{hh}} + \mathbf{b}_{\text{h}}),$$



HIDDEN STATE



$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t.$$

GRU PYTORCH

```
rnn = nn.GRU(10, 20, 2)
input = torch.randn(5, 3, 10)
h0 = torch.randn(2, 3, 20)
output, hn = rnn(input, h0)
```

Inputs: input, h_0

- **input**: tensor of shape (L, H_{in}) for unbatched input, (L, N, H_{in}) when `batch_first=False` or (N, L, H_{in}) when `batch_first=True` containing the features of the input sequence. The input can also be a packed variable length sequence. See `torch.nn.utils.rnn.pack_padded_sequence()` or `torch.nn.utils.rnn.pack_sequence()` for details.
- **h_0**: tensor of shape $(D * \text{num\_layers}, H_{out})$ or $(D * \text{num\_layers}, N, H_{out})$ containing the initial hidden state for the input sequence. Defaults to zeros if not provided.

where:

$$
\begin{aligned}
N &= \text{batch size} \\
L &= \text{sequence length} \\
D &= 2 \text{ if bidirectional=True otherwise } 1 \\
H_{in} &= \text{input\_size} \\
H_{out} &= \text{hidden\_size}
\end{aligned}
$$

Outputs: output, h_n

- **output**: tensor of shape $(L, D * H_{out})$ for unbatched input, $(L, N, D * H_{out})$ when `batch_first=False` or $(N, L, D * H_{out})$ when `batch_first=True` containing the output features (*h_t*) from the last layer of the GRU, for each *t*. If a `torch.nn.utils.rnn.PackedSequence` has been given as the input, the output will also be a packed sequence.
- **h_n**: tensor of shape $(D * \text{num\_layers}, H_{out})$ or $(D * \text{num\_layers}, N, H_{out})$ containing the final hidden state for the input sequence.

11 - INTRODUCTION TO ATTENTION

QUERIES, KEYS AND VALUES

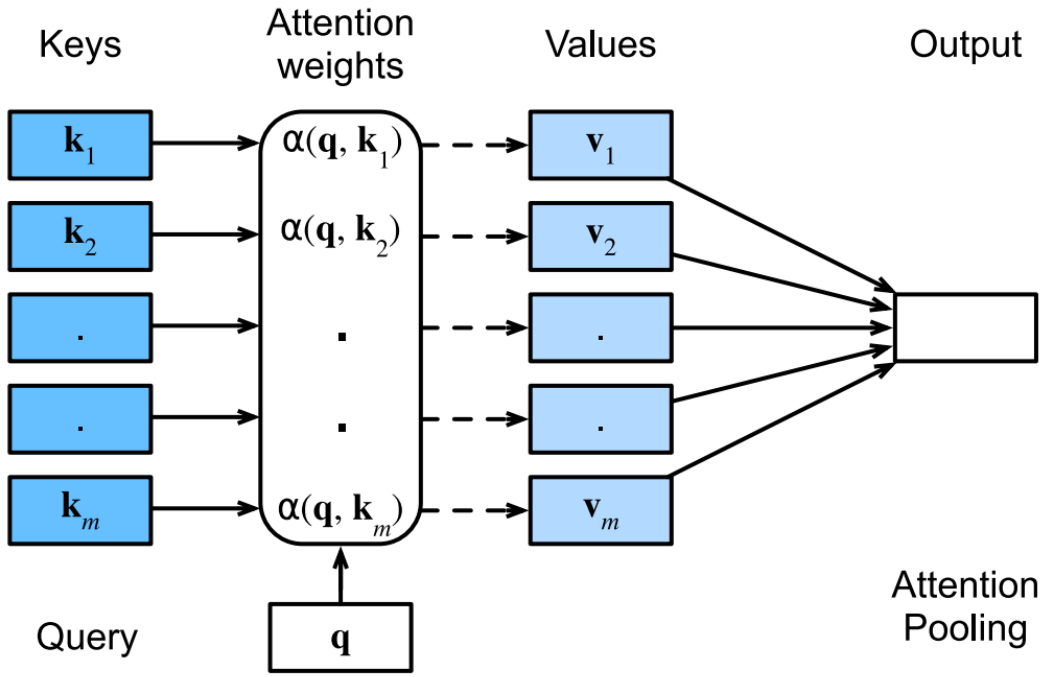
$$\text{Attention}(\mathbf{q}, \mathcal{D}) \stackrel{\text{def}}{=} \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i,$$

ATTENTION WEIGHTS

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\alpha(\mathbf{q}, \mathbf{k}_i)}{\sum_j \alpha(\mathbf{q}, \mathbf{k}_j)}.$$

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_j \exp(a(\mathbf{q}, \mathbf{k}_j))}.$$

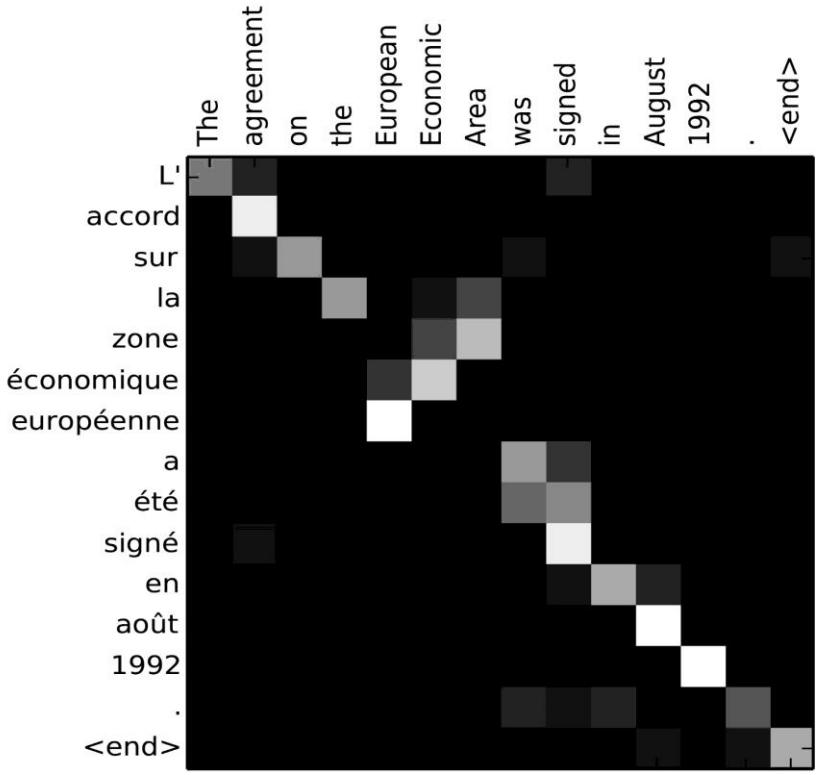
$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=1} \exp(\mathbf{q}^\top \mathbf{k}_j / \sqrt{d})}.$$



MASKED SOFTMAX OPERATION

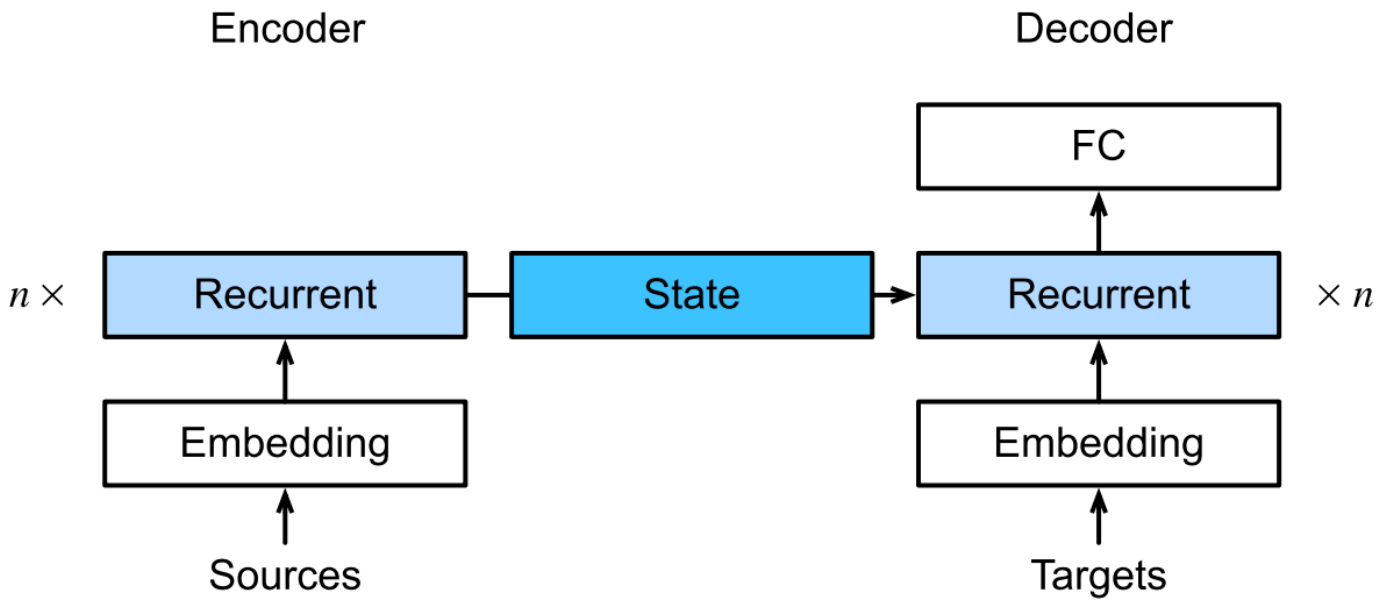
```
Dive  into  Deep     Learning
Learn to    code     <blank>
Hello world <blank>  <blank>
```

VISUALIZE ATTENTION WEIGHTS



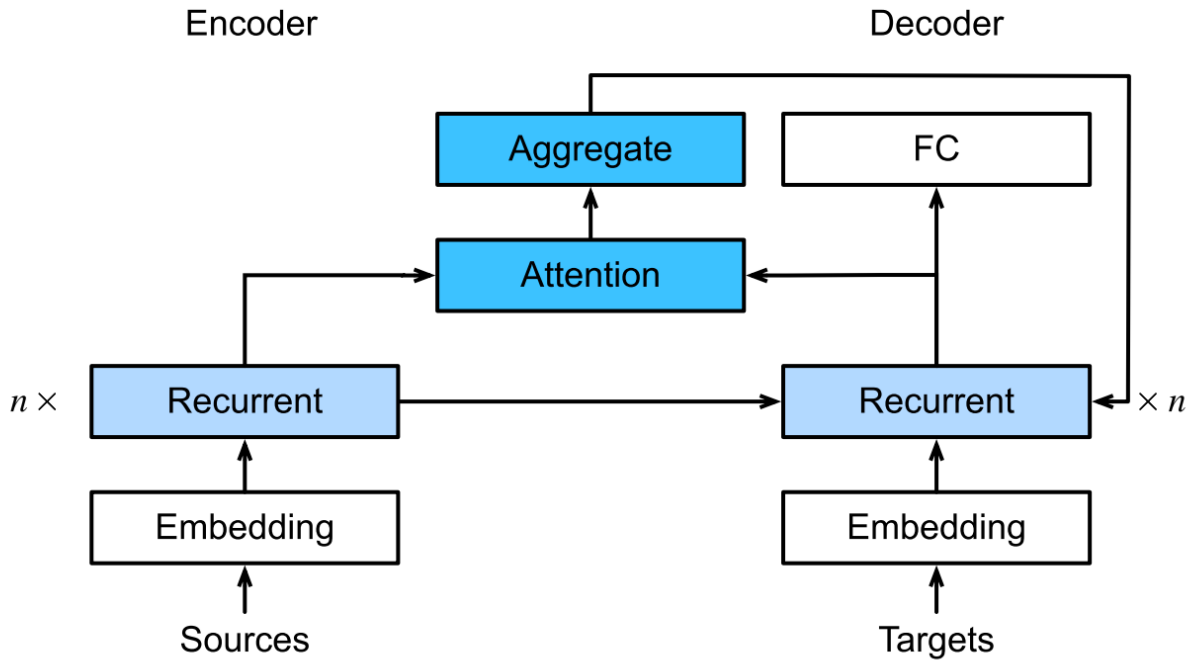
11 - BAHDANAU ATTENTION

SEQUENCE TO SEQUENCE MODEL

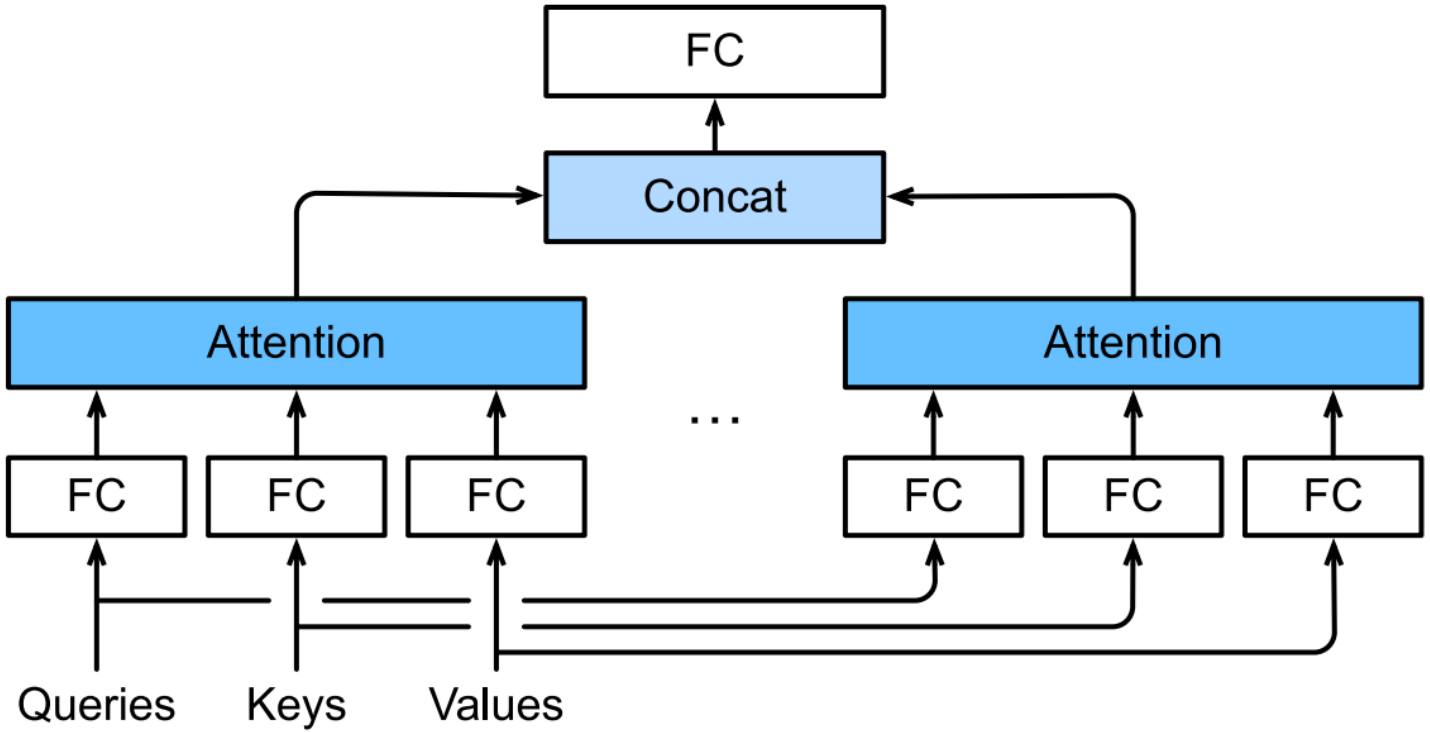


SEQUENCE TO SEQUENCE MODEL WITH BAHDANAU ATTENTION MECHANISM

$$\mathbf{c}_{t'} = \sum_{t=1}^{T} \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t)\mathbf{h}_t.$$



MULTI-HEAD ATTENTION



MULTI-HEAD ATTENTION IN PYTORCH

```
multihead_attn = nn.MultiheadAttention(embed_dim, num_heads)
attn_output, attn_output_weights = multihead_attn(query, key, value)
```

CLASS torch.nn.MultiheadAttention(*embed_dim, num_heads, dropout=0.0, bias=True, add_bias_kv=False, add_zero_attn=False, kdim=None, vdim=None, batch_first=False, device=None, dtype=None*) [SOURCE]

Allows the model to jointly attend to information from different representation subspaces as described in the paper: Attention Is All You Need.

Multi-Head Attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \dots, head_h)W^O$$

where $head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

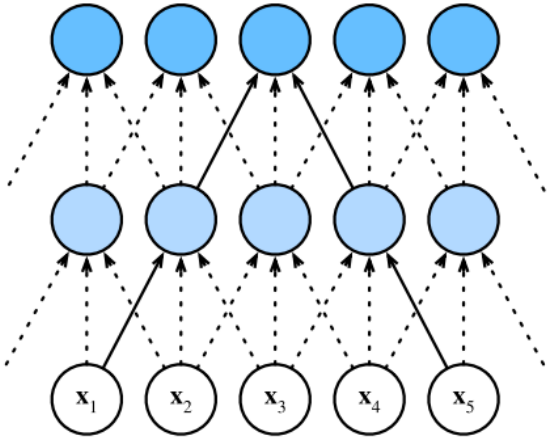
`nn.MultiHeadAttention` will use the optimized implementations of `scaled_dot_product_attention()` when possible.

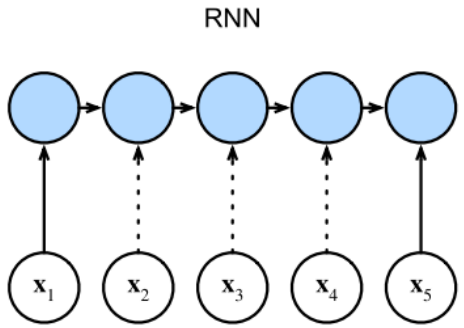
In addition to support for the new `scaled_dot_product_attention()` function, for speeding up Inference, MHA will use fastpath inference with support for Nested Tensors, iff:

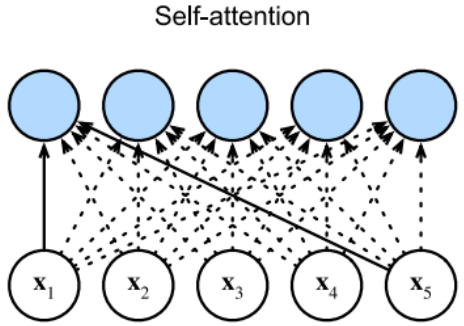
- self attention is being computed (i.e., `query`, `key`, and `value` are the same tensor).
- inputs are batched (3D) with `batch_first==True`
- Either autograd is disabled (using `torch.inference_mode` or `torch.no_grad`) or no tensor argument `requires_grad`
- training is disabled (using `.eval()`)
- `add_bias_kv` is `False`
- `add_zero_attn` is `False`
- `batch_first` is `True` and the input is batched
- `kdim` and `vdim` are equal to `embed_dim`
- if a NestedTensor is passed, neither `key_padding_mask` nor `attn_mask` is passed
- autocast is disabled

11 - SELF ATTENTION AND BEYOND

CNN VS RNN VS SELF-ATTENTION



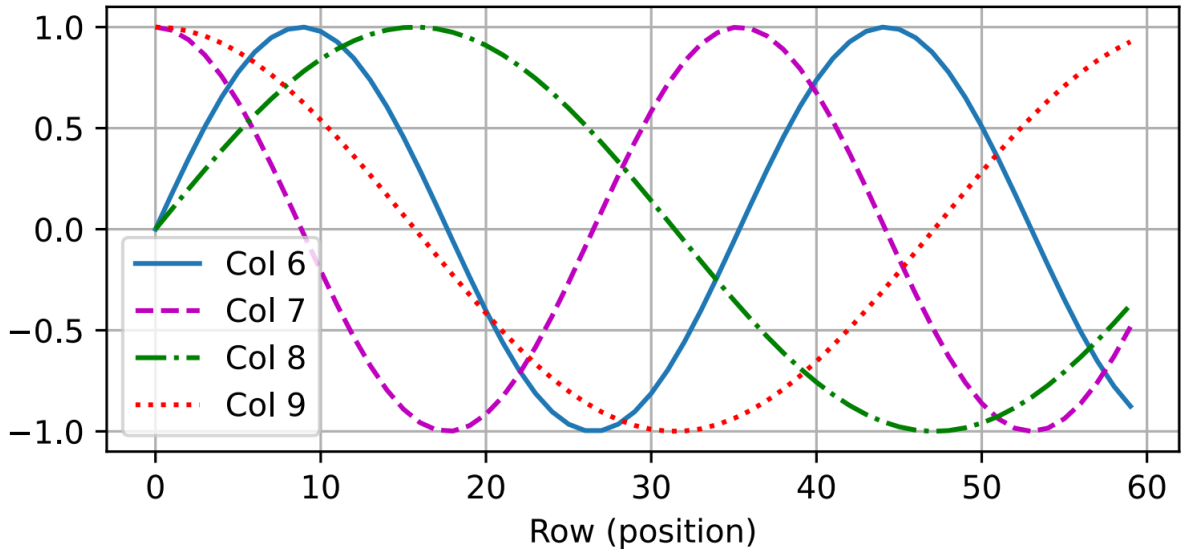




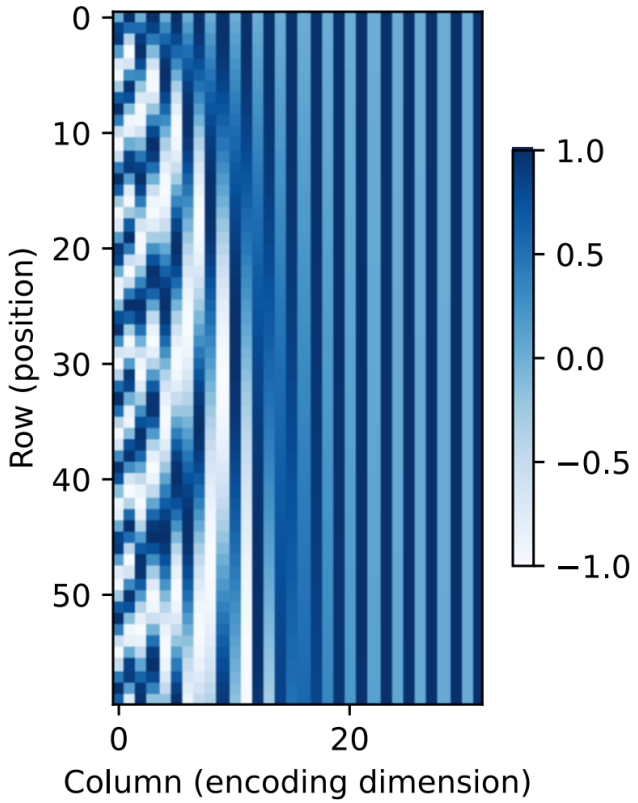
POSITIONAL ENCODING

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right),$$

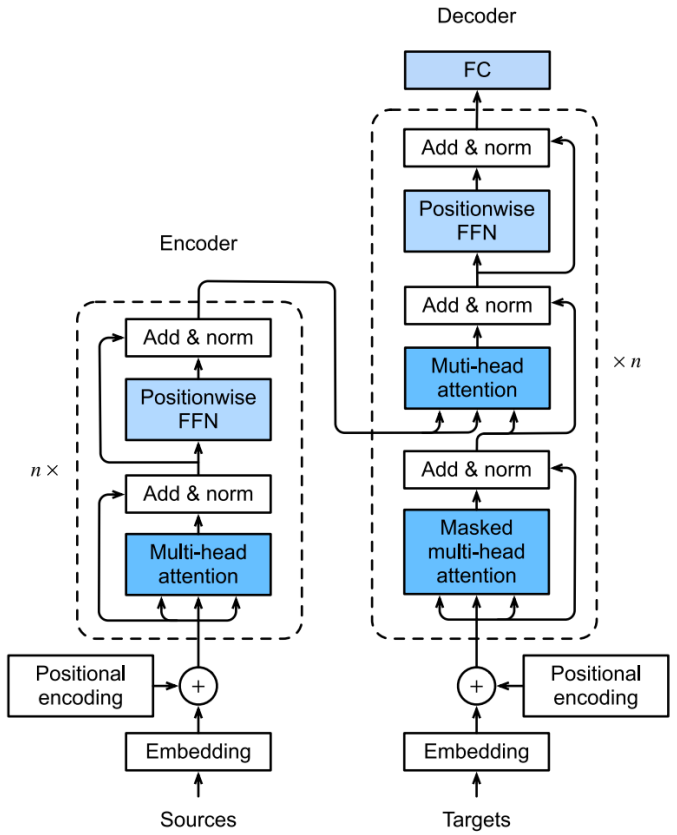
$$p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$



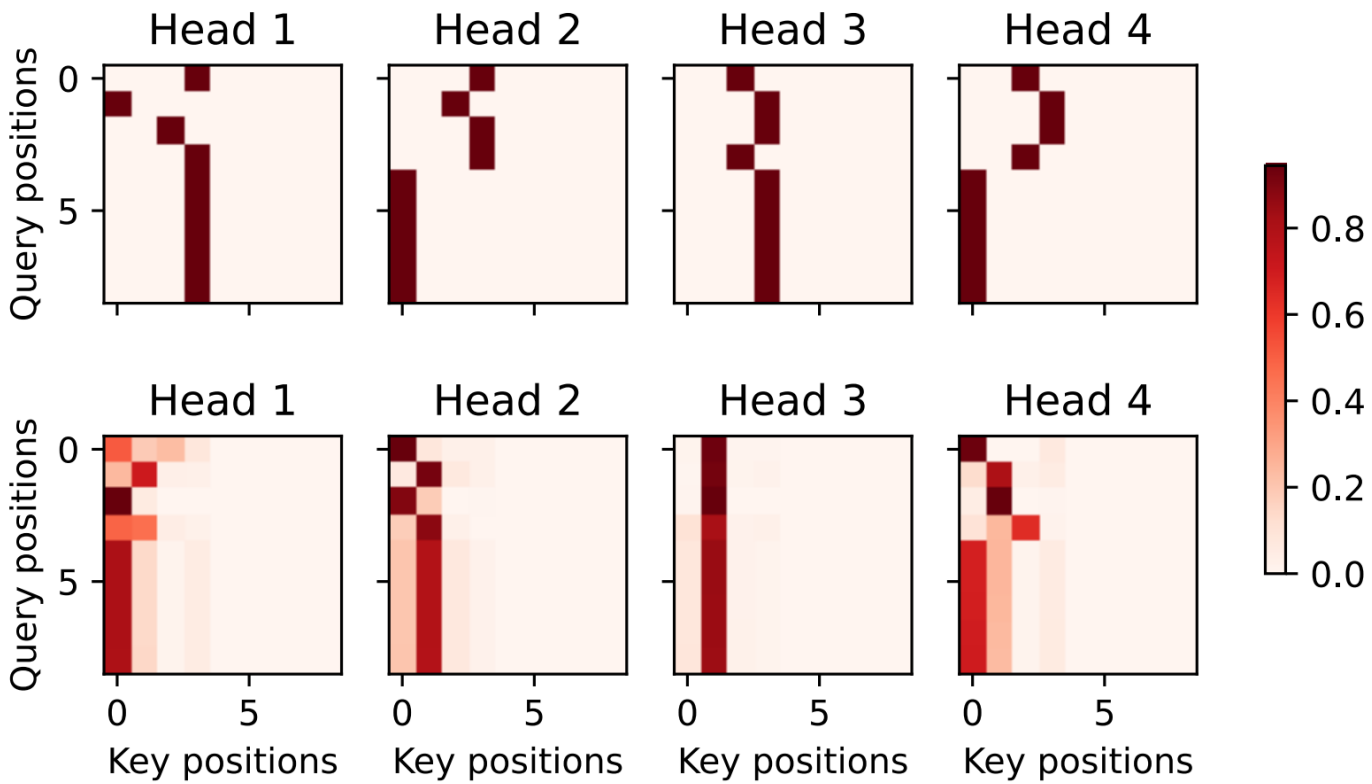
ABSOLUTE POSITIONAL INFORMATION



THE TRANSFORMER ARCHITECTURE



ATTENTION WEIGHTS FROM TRANSFORMERS



TRANSFORMERS IN PYTORCH

TRANSFORMERENCODERLAYER

CLASS torch.nn.TransformerEncoderLayer(*d_model, nhead, dim_feedforward=2048, dropout=0.1, activation=<function relu>, layer_norm_eps=1e-05, batch_first=False, norm_first=False, bias=True, device=None, dtype=None*) [SOURCE]

TransformerEncoderLayer is made up of self-attn and feedforward network.

This standard encoder layer is based on the paper "Attention Is All You Need". Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in Neural Information Processing Systems, pages 6000-6010. Users may modify or implement in a different way during application.

TransformerEncoderLayer can handle either traditional torch.tensor inputs, or Nested Tensor inputs. Derived classes are expected to similarly accept both input formats. (Not all combinations of inputs are currently supported by TransformerEncoderLayer while Nested Tensor is in prototype state.)

If you are implementing a custom layer, you may derive it either from the Module or TransformerEncoderLayer class. If your custom layer supports both torch.Tensors and Nested Tensors inputs, make its implementation a derived class of TransformerEncoderLayer. If your custom Layer supports only torch.Tensor inputs, derive its implementation from Module.

TRANSFORMERENCODER

CLASS torch.nn.TransformerEncoder(*encoder_layer, num_layers, norm=None, enable_nested_tensor=True, mask_check=True*) [SOURCE]

TransformerEncoder is a stack of N encoder layers.

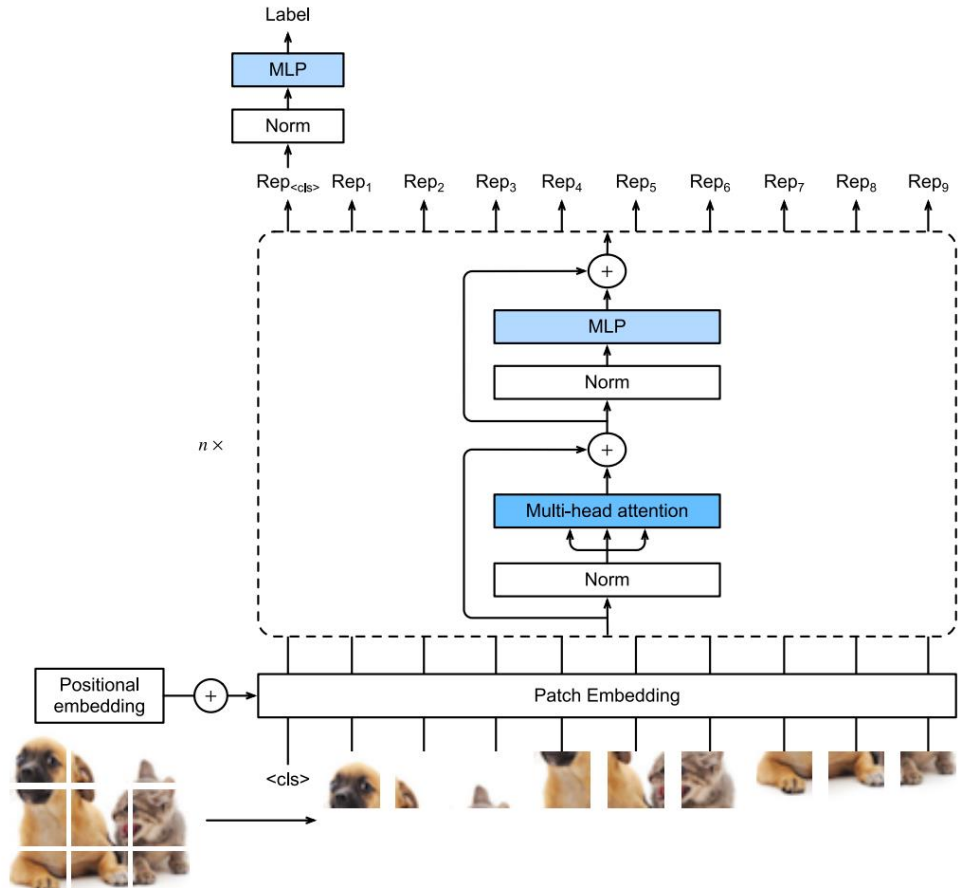
Users can build the BERT(https://arxiv.org/abs/1810.04805) model with corresponding parameters.

Parameters

- **encoder_layer** – an instance of the TransformerEncoderLayer() class (required).
- **num_layers** – the number of sub-encoder-layers in the encoder (required).
- **norm** – the layer normalization component (optional).
- **enable_nested_tensor** – if True, input will automatically convert to nested tensor (and convert back on output). This will improve the overall performance of TransformerEncoder when padding rate is high. Default: `True` (enabled).

```
encoder_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8)
transformer_encoder = nn.TransformerEncoder(encoder_layer,
num_layers=6)
src = torch.rand(10, 32, 512)
out = transformer_encoder(src)
```

VISION TRANSFORMERS



REFERENCES

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into deep learning. Cambridge University Press.
- https://www.statology.org/interpolation-vs-extrapolation/
- Lillicrap, T.P., & Santoro, A. (2019). Backpropagation through time and the brain. Current Opinion in Neurobiology, 55, 82-89.